

Course Overview

Course Overview

Hi everyone. My name is Filip Ekberg, and welcome to my course C# Advanced Language Features. I'm a Principal Consultant and CEO at a consultant agency operating out of Gothenburg in Sweden. I focus on building fast, powerful, and easy-to-maintain solutions. This course is for those of you that want to build reliable and powerful applications. We'll cover all the topics that you need to understand to effectively introduce the advanced language features of C# in your applications. This includes understanding how delegates, lambdas, extension methods, and events work, how to use anonymous types and tuples, how to express yourself with pattern matching to build more understandable and maintainable code, creating record types and deconstructing objects, how to approach nullability and avoid crashes due to null reference exceptions, defining ranges and slicing arrays efficiently. And finally, you will learn about garbage collection, avoiding memory leaks, and much more that will help you build better applications in C#. By the end of this course, you'll know how to best approach the advanced language features of C# and where they fit in your application. Before beginning the course, you should be familiar with the fundamentals of C#. I hope you will join me on this journey to take your application development to the next level with the course C# Advanced Language Features at Pluralsight.

Introduction

Introducing Advanced Language Features in C#

C# as a programming language has evolved through the years. And with each version, language features have been improved and new ones have been added. A lot of the language features in C# are heavily used throughout all different types of applications, some of them a bit more advanced than others. It can be puzzling to understand how they're best applied in real applications. But don't worry. You are in good hands. Throughout this course, you will learn all about the very important and

advanced language features of C# and how they fit in your applications. No matter if you are new to C# or an experienced C# developer, you will benefit from this course. My name is Filip Ekberg, and welcome to the course C# Advanced Language Features. Throughout this course, you will explore the different language features and apply these in an application. The sample application that you will be extending is a warehouse management system. You will be replacing, extending, and adding functionality to understand exactly how each language feature is best used. This course will teach you about delegates and lambdas, events, overloading, operator overloading, and extension methods, anonymous types, tuples and deconstruction, pattern matching, record types, nullable reference types, indexers, ranges, and indices. And finally, you will learn about garbage collection. Before we jump in to the first language feature, let's look at the sample application we'll be working with throughout this course. You can easily follow along with the demos and exercises in this course. The course page includes a link to download all the exercise files. Download the ZIP containing all the files that you need for this course. These can also be downloaded from GitHub, which will ensure that you have the latest version of the exercise files. Extract the files wherever you want. I like to put my files in C:\Code as a too long path can often cause problems. This is a simple console application that I've opened in Visual Studio. The first thing you'll notice is an empty Program.cs file. If this is the first time you see the new console template, you might be wondering where the main entry point is. This here is a top-level statement, and it's the default way of building your console applications or minimal APIs in .NET 6 and forward. This concept was introduced in the C# Fundamentals course. A compiled top-level statement will, in fact, generate the traditional code that you see in your program file. The solution also has a few other projects, one library that contains the business logic of this application to handle, for example, how an order is processed or how it's shipped. Another library in this solution contains all the domain objects. For example, there is a representation of what an order is. An order has an order number, items that have been purchased, and the preferred shipping provider. You will be doing all your work in these different projects to explore the different types of language features. If you run into any problems, feel free to reach out on the discussion on the Pluralsight website and we'll make sure that you can follow along with this course. While taking this course, feel free to pause the video at any time and add the code to your project and fill out the exercise files. This way, you will learn a lot more about all the different language features and how they work in C#.

Version Check

This course was created using the latest version of C# and .NET. No preview versions or preview features have been used in the course. I'm using the latest version of Visual Studio on Windows. There is a free community edition available. Optionally, you can follow along in your favorite development environment on your preferred operating system. While this course uses the latest version of C# and .NET, many of the language features covered are applicable to older versions of C#, ranging all the way back to C# 1. If you use an older version of the C# compiler, I cannot guarantee that all the features work as shown in this course. If you happen to run into any issues or have questions about backwards compatibility, leave a question on the Discussion tab on the website. All the features and principles covered in this course can be applied in any type of .NET application, no matter if it's a console application, Windows application, or ASP.NET Core website. Proceed to the next module to learn about the very useful and important language features, delegates and lambdas.

Delegates and Lambdas

Introducing the Delegate

Wouldn't it be great if there was a way to reference a method and be able to invoke that method without it being a part of the current scope? Welcome to the module where we are going to talk in detail about using delegates and lambdas. You're watching the course C# Advanced Language Features, and I'm Filip Ekberg. A delegate in C# looks and acts very much like a traditional function pointer that you may have seen or heard about from other programming languages. A traditional function pointer is exactly what it sounds like, a way to point to a particular method. You could have a method that accepts a function pointer so that you have a way to extend its behavior or receive a callback when that method completes. In C#, a function pointer is known as a delegate. However, out of the box, the C# delegate provides a bit more functionality. Throughout this module, you will learn about the delegate keyword, action, action of T, func, func of T, and lambdas. First, let's go through the delegate. Think of a delegate as a way to represent the required method signature. Whenever you define that you want to use a delegate, the consumer of the delegate knows exactly what the function

you will point to will return, as well as what the expected parameters are. You can construct a delegate by introducing the delegate keyword. You then proceed to declare the method signature. For example, here's a delegate that returns an order instance and has two parameters. One of them is item, and the second parameter is an integer representing the quantity. Nothing in this declaration defines what the method will do with its parameters or what result it will return. These two different methods both match the signature defined by the delegate. One defines that it returns an Order, while the other returns a subclass type called ProcessedOrder. The return type supports covariance, while the delegate parameter supports contravariance. This means a method with the return type object wouldn't match our defined delegate as you cannot use a less derived object when matching the return type. On the other hand, since the parameters support contravariance, you can change the item parameter to object and that would work. The delegate can then be referenced throughout the application using its name. This means that whenever we declare that we want to use this defined delegate, both of these methods could be viable references. Whoever uses the delegate doesn't need to care about the implementation. All it knows about is the return type and the parameters. Let's explore exactly how this works throughout this module. Being able to reference and invoke a method without that method being a part of the scope or class that you are interacting with opens up a lot of potential possibilities. In essence, the work is being delegated somewhere else. Let's talk about a few common use cases for delegates. Use it to receive a callback when a certain operation completes. This was a lot more common in the past where you might have executed work on a separate thread. And after that work completed, you wanted to execute a particular method. This is where you'd introduce a callback and pass a reference to that method, also known as using a delegate. In user interface programming such as WPF, WinForms, Xamarin, .NET MAUI to name a few, you very often rely on something called events. This will be thoroughly covered in the next module. But this builds on top of delegates. When you press a button on your user interface, a method should be executed. To implement the handler for that event, you need to correspond to the delegate signature. This method is then being referenced by using a delegate. Another common use case is extensibility. You can provide a way to replace or extend functionality of your classes without requiring them to be subclassed. In our example application that handles everything inside a warehouse, a good example of this is where the order is being processed. This is done inside the business layer and may be used across multiple different applications in the business. These applications may all want to inject functionality to run when the

order starts being processed, as well as when the order of processing completes. This could be used in many ways. One of them could be how you are producing the shipping label after the order is processed or even how the order confirmation is being sent. The exact functionality could be decided and replaced in runtime by simply passing a reference to a method that matches the given delegate. You will notice that some libraries in .NET, as well as some that you install from NuGet, will provide you with the capability to inject functionality. Some even depend on it. This could be everything from determining sort order when sorting an array of elements to callbacks. In this scenario, being able to pass a reference to that method is really handy. Where you decide to use delegates in your applications may, of course, vary. Being able to provide a way to extend or replace functionality at runtime is going to prove very powerful. We will also highlight where this may cause potential problems. As with any language feature, you should completely understand its purpose and the potential problems that you may run into. The tools of the language are here to help you, but use it where it's appropriate and think about how it architecturally fits in your application. With a bit of background and a peek at what a delegate declaration looks like, let's proceed to construct a delegate and evoke that in our application.

Creating, Using and Invoking a Delegate

The warehouse application that we are tasked to work in needs to be updated. We want to provide the consumers of the business layer with a way to run their own code when an order has been initialized and when the order processing is completed. Let's jump into Visual Studio. In the program, I am going to introduce two methods that I want to be invoked when the order reaches two different states. When the order has completed its initialization, I'd like to send a message to the personnel working in the warehouse to prepare them to pack the order. After the entire order has completed its processing, I'd like to send a confirmation email. Both methods are simple. They run some arbitrary test code, and they don't return anything. They also do not have any parameters passed to them. This could be solved in many ways. But I know that the business layer is used by many different applications. So building this behavior into the order processor wouldn't be a great idea. Your preferred architecture may vary. In our case, we are going to achieve all of this without requiring anyone to subclass or further extend the existing implementation. The business layer in the application provides a class that handles processing of an order. It exposes a public method. This will internally call other methods.

One of them initializes the order. We'd now like to produce a way to run code when the order has completed its initialization, as well as when the order process is completed. This will provide both flexibility and extensibility. Imagine if this processor executes in the background. Providing a way to get a callback when an order reaches a certain state allows the consumer to act on that information. This could be to populate the UI with some information about which order was most recently processed. To allow for such a callback, we are going to introduce a delegate. This will allow anyone that uses this class to pass a reference to the method they want to invoke when the order reaches a certain state. Hence, we are delegating the implementation to someone else. The methods Initialize and Process will both try to invoke two corresponding delegates. In other words, they will try to invoke methods that we are simply pointing to but do not belong to this class. Remember the methods from the program file? The idea is that the delegate that we create matches those method signatures.

When the order processor is created, we need to provide a pointer to those methods. This class can then invoke the delegate, which will run the methods we've pointed to. When you create a delegate, you use the delegate keyword. This tells the compiler that you are about to specify the required method signature. After the delegate keyword, you specify the return type. We're not going to return anything at this point. And therefore, we'll specify void. Now it's time to give this delegate a name. Let's call it OrderInitialized. As with all your types, it's good to give them a descriptive and understandable name. We are not going to pass any parameters when we invoke this delegate.

Therefore, the method signature of this delegate is completed. You then use the name of the delegate as you'd use any type. A delegate can be declared inside or outside of a class. And you can, of course, use it with the access modifiers such as public, protected, internal, or private. The delegate must be at least as accessible as the way you expose it. This means you cannot declare a private delegate and expose it through a public method. Let's now use this delegate that we have created. You can introduce a property of type OrderInitialized. We call it OnOrderInitialized. Let's expose both a public getter and a public setter. This property could now point to any method that corresponds with the method signature that we defined when declaring the delegate. The method you point to does not have to use the same name as the delegate. It only needs to match the method signature. Both methods defined in the program would be viable options as they have matching signatures before we get to that part, let's look at how to use the delegate to invoke a referenced method inside this class. Since the property is a delegate, which ultimately is just a referenced method, could we use this as a

normal method call? Certainly. As you can see, if we call `OnOrderInitialized` at the end of the `Initialize` method, this looks like a normal method invocation because that's exactly what it is. And the whole point of the delegate is to allow it to point to a method that lives somewhere else and might not even be known when this portion of the code is compiled. An external library or application could be setting this up when they consume this business layer. Without doing anything else, let's head over to the program and make sure that we create an order, the `OrderProcessor`, and then call the `Process` method. What do you think will happen at this point? We haven't set `OnOrderInitialized` to anything, so what will this method call do? You will get a null reference exception. If you think about it, it makes sense. The property hasn't been set to a value. Hence, the method reference is null. And therefore, you get the null reference exception. As with any reference types, you can guard against this using an `if` statement to check if the property is null before you invoke the delegate. Instead of doing that, let's explore if there are any available methods on the delegate. There are a few methods and properties available because when you use the `delegate` keyword, under the hood, the compiler will use the type `delegate` or `multicast delegate`. There's one method that we want to use. That is the method `Invoke`, which will allow you to invoke the delegate. You can also see `BeginInvoke` and `EndInvoke`, which are old ways of asynchronously invoking a method. Nowadays, you would use the task-based asynchronous pattern, which is thoroughly covered in my course about asynchronous programming. With that being said, you'll want to use the method `Invoke`. Are you wondering why? Since this is a method call on a type, you could use the null conditional operator. This will make the code cleaner and easier to read. We'll talk more about avoiding null reference exceptions and other ways to approach this later in the course in the module `Nullable Reference Types`. If you rerun the application, there is no longer going to be a null reference exception. Surely that's a better experience. The `OrderProcessor` is now exposing a public property that'll allow you to reference a method that will be executed when the order has been initialized. We can go to the program and set the property to point to the method `SendMessageToWarehouse`. This can be done using normally property access or the object initializer. Keep in mind that you do not add the parenthesis because this is not a method call. You are now pointing to the method to delegate the work. Let's rerun the application with the debugger attached and see how this works. As you step into the method process and then into `Initialize`, you can see that the property on `OrderInitialized` is referencing the method. When you step into the invocation, you should end up inside the `SendMessageToWarehouse` method. You've successfully delegated the

work to a method that wasn't a part of the processor. An important note to make here is that since this delegate is a public property and it's really just a pointer to a different method, anyone that has access to this property could invoke the method. This could be seen as one of the drawbacks of publicly exposed delegates. There's no rule as to who owns the invocation. You could simply invoke the delegate like any other method on the `OrderProcessor` instance from anywhere in the application. Let's add one more delegate. Create a delegate that returns void called `ProcessCompleted`. Because this is used like a normal type, you could add this as a parameter to the method process. I'm going to make it optional and set it to the default value of null, then call this at the appropriate time. The second parameter to the method process now accepts a delegate. As opposed to exposing the delegate as a public property, this approach allows you to control that this delegate is only invoked from this class, which makes a lot of sense. We most certainly wouldn't want this delegate to be executed when there, in fact, hasn't been an `OrderInitialized`. Exposing that delegate publicly would, however, allow anyone to invoke it. Going back to where the method process is being called, we can now pass a reference to a method. Just as before, simply write the method name that you want to reference. As you run the application, both delegates should be invoked, and you've successfully introduced a way for anyone that uses this class to run code in the processing chain without having to do any further extensions of this class. All they must do is pass a reference to a method. There's still a lot to learn about delegates. Next, we are going to explore how to return values and how to add parameters.

Delegates with Return Values and Parameters

The next logical step is to look at how you could return a value from a delegate or even how to use parameters. In our example, we had two methods, which in reality would depend on the order to successfully operate. The method that tells the warehouse to get ready and the method that sends a confirmation email would both need to use the order to know what was processed. Ideally, we'd like to get the order passed as a parameter to these methods. To allow a value to be passed as a parameter to a delegate, the method signature needs to be changed. Head over to the `OrderProcessor`. We can change both of the delegates to indicate that they require a parameter to be passed to them. Now, both the `OrderInitialized` and `ProcessCompleted` both specify that methods that correspond with this delegate need to require an order to be passed as the first parameter. We will now get a few errors in the solution because we are no longer invoking the delegates in the correct way. As you'll see on the

Invoke method call, it requires the order to be passed. This is a quite simple fix. All you must do is pass the order into the invocation just like you would if you called the methods directly. Let's just take a moment to appreciate what's going on here. We've defined a required method signature for what really is a function pointer. This class expects this function to be referenced. And as long as it's not null, it will invoke a method that it knows nothing about, doesn't know more than only the method signature.

When the application runs and this code path is executed, it will delegate the work to that referenced method. We are passing the order that we are currently processing into both of these delegated method calls. If you try to run the application, you'd notice that we still have a couple of errors that we need to attend to. The method signatures in the program does no longer correspond with the delegate that we have defined. Again, this is a simple fix. The method that you reference needs to have the exact same method signature as declared by the delegate. Thus, add the order as its first parameter to both of these methods. Now, the error has disappeared. This lets you use the order that is passed from the OrderProcessor and act accordingly on that. You can see that you didn't have to change anything where you pass the method as a reference. All you need to make sure of is that they have the correct method signatures. This approach provides flexibility as you can introduce functionality that shouldn't live inside the order processor. As I mentioned earlier, there are other patterns and practices and many ways to achieve a similar extensibility. At the end of the day, it all comes down to what you prefer. One thing to keep in mind is that when you invoke the delegate, it's like any normal method call. If you throw an exception, for example, it's not going to automatically be caught. You'd have to handle that just like you'd do with any other method call. For example, you could wrap the invocation in a try and catch block. All this approach is doing is delegating a method implementation to the external consumers. If you want to rely on the outcome of a delegated method, you'd most definitely want to use a return value. Just like adding a parameter, adding a return type is just as simple. You'd go ahead and change the delegate declaration to your preferred method signature. This can return absolutely anything. Change the OrderInitialized delegate to return a Boolean. Then we are going to slightly change how we invoke this. If we don't have an instance of the delegate, we'll do nothing. If the invocation returns false, something went wrong with the initialization of the order. We can then throw an exception. While unlikely, I do want to highlight a potential race condition. If you use the null conditional operator, you don't have to worry about this at all. This is because when you use the null conditional operator, it will automatically create a local variable that represents the delegate. After that,

it performs a null check. And then, it invokes the delegate through that local reference. Why does this matter, you might ask. Delegates are immutable just like strings are. To change them, you have to point to a new reference. This is why storing the reference inside the method that performs the null check and then performs the invocation prevents a race condition because that locally captured reference cannot be changed at any time while this method is being executed. As long as you use the null conditional operator, this will be handled for you. If you compile the application, we again get a compiler error that tells us that we need to update the method signature in the program. Change `SendMessageToWarehouse` to return `false` and run the application with the debugger attached. You can see that we get into this method, it returns `false`, and we'll get an exception thrown. You've successfully added both a parameter and a return value. You can change this to return `true` instead and then rerun the application. I'd suggest that you experiment on your own with what we've just learned. As another fun experiment, you can go ahead and try what happens if you return a delegate from a method. Since a delegate is used just like any reference type, you can specify that the return type is a delegate as well. Simply return a method reference that corresponds with that delegate, and whoever uses this method will be able to invoke the delegate that is returned. You can then use this returned delegate and invoke that. You could call `Invoke` or use it like a normal method. Personally, I prefer using `Invoke` especially in this case as it would make it a little bit more readable. I'd really recommend that you pause the video and experiment with delegates on your own. Please remember that if you run into any issues, I'm here to help. Reach out on the discussion board at any time. Next, we are going to be looking at how you can construct a chain of delegates and invoke multiple methods with a single delegate reference.

Multicast Delegate and Chains

A huge benefit to delegates is that it supports multicast delegates out of the box. This means that you could construct a chain of delegates that will be sequentially executed when you invoke the referenced delegate. Let's say that you define three methods that all correspond with the `ProcessCompleted` delegate. To keep it simple, use an expression bodied member and simply print something to the console, this to understand the execution order. We can name them `One`, `Two`, and `Three`. How could we indicate that we want all of these methods executed in a chain whenever `ProcessCompleted` is invoked? This is where we want to construct something known as a multicast delegate. Start off by

specifying that you want to create a `ProcessCompleted` delegate. As you've seen previously, you can use this like any normal variable. Since the delegate is declared inside the scope of the `OrderProcessor`, if you want to explicitly use this type, you can write `OrderProcessor.ProcessCompleted`. Call this local variable chain. The first method we are going to reference in our chain of delegates will be the first method called `One`. This just looks like a normal instantiation of a delegate variable. How is this different from what we've done before? So far, there's no difference. We've just pointed to the first method. Can you guess how you'd append a method to construct a chain? Just like any other addition, you can use `+=`. When you write `chain +=` followed by another method, it will now change what the chain variable points to, and it will be a multicast delegate where you have a chain of two methods. It uses the same delegate type, `ProcessCompleted`. To anyone that expects this delegate, it looks and acts just like any other delegate that you are using. The consumer of the delegate does not have to care or do anything differently to execute all the methods in the chain. Once the delegate is invoked, it will automatically run all the methods in this chain, sequentially that is. This means one method will be executed after the other. It won't do any parallelism for you. Append the second and third method to the chain and pass this to the `Process` call. If you execute the application, you should see the execution order. It'll start by executing the first, then the second, and then the third. You've successfully introduced a chain of method calls using a delegate. There's one thing I'd like you to keep in mind. Remember earlier where we talked about the drawbacks of using a publicly exposed delegate? If you can simply append more methods to a chain like this, a publicly exposed delegate could have methods appended to it from multiple different places. Just as it's easy to add methods, you can also remove method references from the chain. Again, if it would have been a publicly exposed property, anyone could have modified the chain. To remove a method from the chain, let's say the second method, can you guess what you write? You simply subtract with that method. This will remove the delegate from the invocation list. After the subtraction, we'll call `Process` again, then run the application. This should now show you two results from calling `Process` where the second one has one less method invoked through the delegate. You may be wondering now what happens if you try to add the same method multiple times or even remove a method that doesn't exist in the invocation list. How about we change all the methods in the chain to point to method `One`? Then after the first call to `Process` where we remove one method from the chain, let's keep that code that removes the method `Two` from the chain and then also remove

method One from the invocation list. Before running the application, do you think that the first method will be executed once? And what do you think happens when you subtract the method Two from the list of delegates? Let's run the application to find out. First of all, it's added and executed the method three times. This is exactly what we've indicated in the code. I'm really happy that the application didn't get any exceptions or issues when trying to change the chain to remove a method that was not in the invocation list on the delegate. What's also interesting and good is that when you subtract the method, it only removed a single occurrence. If you had wanted to remove all of them, you'd have to subtract the method three times. When constructing the chain, you may notice that you cannot simply do it in a single line. If you were to construct a `ProcessCompleted` delegate chain and write `One plus Two plus Three`, this will give us an error. You can only append a reference to a method when the left-hand side is a delegate. To get around this, you could cast the method reference to a delegate and append all the methods you'd like after that. This is just another way of constructing the chain. With an understanding of delegates, how a delegate is constructed and invoked, you are now ready to move on and look at lambdas, as well as the `Action` and `Func` classes.

Introducing Lambdas and Anonymous Functions

When you call a method that expects a delegate to be passed to it, in some situations, it would be great if there was a way to define that method inline. There certainly is. You may have heard about something called a lambda or anonymous function. This is a concept that exists in both mathematics, as well as in computer science. As a really simple example, consider that we want to express a function that accepts an order as a parameter and returns true or false depending on if the order has items added to it or not. C# provides two ways of expressing this using a lambda. The result of expressing a lambda will be an anonymous function, which you use like any delegate. The basic pattern to follow is that on the left-hand side you express the expected parameters within parenthesis. You then use the arrow syntax, also known as the lambda operator. And on the right of the arrow, you have two ways of defining the body of the lambda. Either you use an expression or a statement. These two alternatives are known as lambda expression and lambda statement. Using an expression is a simple way to define a one-liner. This should be kept short and neat to make it easy to understand. A statement looks much like a method body. This is what you would use for more complex lambdas where a bit more happens inside the body of the anonymous function. The C# compiler will

do its best to determine all the types used in the lambda. If it can, it will infer the types for the parameters based on where you use it. It will also determine the correct return type. You can explicitly define these yourself. Let's look at this in our application. Remove most of the code that creates the chain. Instead of using the methods that we've defined earlier, we are going to create a chain using lambdas. To create an anonymous function for the `ProcessCompleted` delegate, you need to express a lambda that accepts a parameter and that doesn't return anything. For anonymous functions that do not return any value, you have to use the lambda statement or point to another method that returns void. This is because the whole idea behind an expression is that it yields a result. You start your lambda statement with the expected parameters. This is done with a method-like syntax. You can name the parameters whatever you want. All that matters is that the number of parameters is a match to the delegate. If you explicitly declare that the parameter is of a type that cannot be derived from the delegates, you will get a compiler error. It's better to let the compiler figure out the type for you where possible. Even though the parameter is only used inside the scope of this anonymous function, it's wise to give it a good and descriptive name. Add a console output to the anonymous function. Now that we've defined the first delegate in the chain using a lambda, we can try and append a second one. Just like earlier, you can use addition. If you run the application, you can see that it does indeed run the anonymous function without having to create new methods that are available throughout the class. You can simply pass along an anonymous function, which sole purpose is to cater for the delegated functionality. Look at this. There's one thing that immediately comes to mind that is going to be different from what we could do before. How would you remove one of these delegates from the chain? It's clearly not as simple as subtracting the method as the anonymous function isn't available to reference. While lambdas make it extremely simple to express functionality passed to methods that require a delegate, this is one of its drawbacks. We'll talk more about this in the next module where we'll look at events as it's something you should really keep in mind. Let's completely remove the chain and instead pass only one delegate expressed using a lambda to the `process` method. This would mean that we can completely remove the `sendConfirmationEmail` method. We've now reduced the amount of code in the program, and we've maintained the functionality. Of course, if the anonymous function would have been a bit bigger than just a handful of lines, it would have been wise to extract that to a separate method. Use this to reduce complexity and increase readability. Now that we've seen the lambda statement, let's give the lambda expression a try. For the `OnOrderInitialized`

delegate that we pass when the `OrderProcessor` is instantiated, we can replace this method reference with an inline anonymous function that we declare using a lambda expression. Just as with the lambda statement, you start with the parameters that are passed into this anonymous function. And then, on the right-hand side of the arrow syntax, we create the expression. This delegate, as you may recall, requires a `Boolean` to be returned. As an example expression, we can check if the order has been marked as ready for shipment. If it hasn't, this expression will return `false`. You could also construct more complex expressions. It could even involve method calls. As a word of advice, I would recommend that you keep it as simple as possible. And if it gets too complex, you can always extract portions of that into separate methods. I want you to take a moment to think about what we've just done here. We changed the delegate. Instead of sending a message to the warehouse, it instead checks on a completely different property. This means that the `OrderProcessor` and how it handles the order is now completely different from the earlier implementation. This really highlights the usefulness of delegates, lambdas, and anonymous functions. Applications may share access to this business layer that exposes the `OrderProcessor`, and they could all supply their own logic for how to handle this delegated logic. As mentioned earlier, the compiler will infer both the return type, as well as the parameters. But you can explicitly define all of these if you'd like to. The return type is defined right before the parenthesis where you declare the parameters. And the parameter types, well, they're defined together with the parameters. Another big benefit of using anonymous functions is that it can capture variables from the current scope. If you introduce a variable in the program, you could use that in one of these lambdas. This is a huge benefit over simply pointing to a different method. When you capture a variable, it's worth to mention that it will only be disposed when the delegate is no longer used and is marked for disposal. If you want to ensure that the anonymous function do not introduce any side effects in terms of changing variables in your scope or even being able to capture those variables, you can declare this as `static`. This acts just like a normal static method. You could, of course, capture static variables in the current scope.

Lambdas in Minimal APIs

I'd like to take a look at a completely different example. Lambdas are heavily used in the new concept of minimal APIs. Consider this very small top-level statement. This is what you'd get when you type `dotnet new web` in the console. The purpose of this application is to give you a very simple web

application. This course is not on web programming using ASP.NET, so don't worry too much about the get, post, and all the details of this application. This is just an example of what is known as a minimal API. This approach heavily relies on using delegates and lambdas. There's some code in here that gives you a web application through a builder. Then the app is what you use to map functionality to different endpoints that handles web requests. A very simple example of this is to allow you to map the root of a website and then use a lambda expression to declare what happens when this path is requested. If I run this application, it spins up a website that returns the data from that anonymous function. Just as that, you've created your first API in ASP.NET. This is a good opportunity to talk about using attributes. Just like with other methods, you can use attributes with lambdas. In this case, I'd like to use a very specific attribute just to show you how this works. If I add a parameter to the anonymous function and append the attribute FromHeader, this would indicate that the value is populated from one of the request headers that matches the name of the parameter. In this case, the accept header. Running this example application shows that we got the accept header mapped into this parameter, and we can use this within this anonymous method defined by our lambda. When a delegate is expected, a lambda provides a very nice way of defining that method. Throughout your work in C#, I can guarantee that you will be using lambdas, expressions, and delegates quite a lot, especially when you encounter the language integrated query also known as LINQ. Consider a generic method. Because it's generic, the method that filters through the list doesn't necessarily know about the type. Therefore, it's difficult for such a method to determine equality. This is where delegating that comparison to somewhere else is clearly a beneficial approach. You could use a lambda expression to define that filter. This is yet another great example of where delegates, lambdas, and the anonymous functions are truly useful. Next, we are going to look at making it simpler and more convenient to declare that we expect a delegate by using Action and Func.

Using Action<T> and Func<T, TResult>

When we first introduced the delegate, we had to create the method signature. This does, of course, support generics. This means that you could construct a delegate that would be reusable throughout your library. What would a generic delegate look like that expects a single parameter but doesn't return anything? You'd start by introducing the delegate keyword followed by the method signature. Let's call it Action because that's really what this is. This method would be generic, and the first parameter

would use this generic type. To create a method that corresponds with this generic delegate, this could be done using a normal method or even a lambda. On the other hand, if you would like to create a generic delegate that allows you to also return a value, you'd have to construct a delegate that returns a `TResult`, which is a generic type. And to distinguish this from a simple `Action`, we can call it `Func`. This generic delegate now uses two generic types, one that determines the parameter type and one that determines the return type. We could construct a method that corresponds with this delegate as well just as with our other generic delegate. Doesn't it feel like this is something that would have been great to get out of the box with .NET? Well, Microsoft felt the same way. And I'm really happy to tell you that there's already an existing `Action` and a `Func` out of the box. There are, in fact, quite the number of overloads for both `Action` and `Func`. `Action` supports up to 16 generic parameters, and none of the `Actions` return a value. `Action` is simply to be looked at as a void delegate. `Func`, on the other hand, has 17 different overloads. This is because the `Func` uses only one generic parameter that determines the return type. Internally, these are all built using the delegate. Just as you saw earlier in this clip, if you create a `Func of Order and bool`, this means that it can reference a method that returns a `Boolean` and accepts an `order` as its first parameter. This could have been the `SendMessageToWarehouse` method that we saw earlier in this module. The local variable that is now of type `Func of Order and bool` is just a delegate that point to the method `SendMessageToWarehouse`. With this in mind, do you think we could refactor the `OrderProcessor` to use `Action` and `Func` instead of the delegates that we created earlier? If you'd like to attempt that, I'd highly suggest that you pause the video and give it a go. If you're ready, let's refactor the `OrderProcessor`. First thing we'll do is to remove the delegates. This will give us a few errors that we need to attend to. What do you think the public property on `OrderInitialized` should be? This delegate needs to accept a parameter of type `Order` and return a `Boolean`. Therefore, the suitable class is `Func`. Change the `OrderInitialized` type to `Func of Order and bool`. This takes care of the first error. Notice how you don't have to change anything inside the `Initialize` method. This is because `Func` is really just a delegate under the hood and works exactly like what we've been using throughout this module. It's just a simpler way to declare exactly what we want with a few less characters. This means less code to maintain and an easier-to-understand solution. It's also good that the `Action` and `Func` are well known amongst other C# developers. This helps to communicate the intent of your code. Scrolling down, there's another error. This delegate accepted the order but didn't return anything. This would be a good opportunity to

use an Action of Order. Running the application should yield the same result we saw before refactoring the class. You could even construct chains just like you saw earlier. Everything works the same. It's just a tidier way of expressing the delegate. If you experimented on your own with lambdas, you may have noticed that if you construct a lambda and assign that to a variable using the var keyword, the compiler will do its best to determine the appropriate Func or Action. If you simply do not care about the variable, but still want to create a lambda expression or lambda statement, you can use what is known as a discard. Using the underscore here indicates that you do not care about the variable. I'll create a local variable called `isReadyForShipment` and construct by anonymous function using a lambda. I have to explicitly say that the parameter is of type `order`. Otherwise, it would have been impossible for the compiler to figure out the type. On the other hand, if I explicitly change `var` to a `Func of Order and bool`, it will work as the parameter in the lambda can be inferred from the type. Not having to explicitly declare that the lambda is of a certain Action or Func is also known as the natural type of the lambda. The compiler will do its best to help you. But you can also be specific in those situations where that would be required. I can almost guarantee that you will be seeing a lot of the Funcs and Actions throughout the time you spend coding in C#.

Summary

This module introduced you to the concept of a delegate. This is a type-safe function pointer in C#. It allows you to reference a method that can later be invoked. We introduced the `delegate` keyword. This lets you define a required method signature. You use the `delegate` keyword followed by the method's return type, the delegate name, and finally, the required parameters. You can then use this defined delegate throughout the entire application by using the name. This has a very small memory footprint and allows for great flexibility and extensibility. You can indicate that you want to use the delegate, which will then let you pass a reference to a method that matches the delegate's signature. You saw how to pass a reference to an existing method and also how to construct an anonymous function using a lambda. The lambda can be defined in two different ways. One uses an expression, which is ultimately a one-liner, and the other is a statement that lets you use multiple lines. And you can use the `return` keyword if the delegate requires you to return a value. It's good to remember to keep it simple. If the lambda statement gets too complex, it might be worth to extract that to a separate method to increase readability. A huge benefit of the lambdas are that they make the code easier to

read because the logic may be defined in the same place as the method called, as well as the fact that it can easily capture variables and use them inside the expression or the lambda statement. The compiler will take care of the internals for you and, as efficiently as possible, reduce the memory footprint and complexity. On the receiving end of the delegate, you can invoke this like a normal method. The problem is that since this is a reference type, the reference might be null, and you could get a null reference exception. It's wise to use the null conditional operator together with the Invoke method. This will only invoke the delegate if there is one defined. Don't confuse Invoke with BeginInvoke and EndInvoke as those are no longer recommended to be used. They were used in the past to run the delegates asynchronously. To execute the delegate asynchronously, you'd use the task parallel library instead, which you could learn all about in my course on asynchronous programming. The delegate that is invoked could be a multicast delegate, which means it is a chain of methods that will be executed sequentially. Anyone with access to the delegate could modify the invocation list, either adding or removing existing methods from this list. Another important note of publicly accessible delegates is that anyone can invoke them. Finally, you learned that you can use Action or Func to define a delegate without having to explicitly define the delegate with a method signature. They both support a lot of input parameters, and the Func allows for a generic return type. I can guarantee that you will be using delegates and lambdas quite heavily throughout your applications. If we want to extend the warehouse application with the capability of signaling when certain stages are reached, how would we approach this? Would you suggest that we publicly expose an action for each of the different signals? In reality, what we'd like is for a way to subscribe to an event. Another way of looking at that is if an order placement takes a little bit of time. Is there a way to get notified when that is done? Could we somehow get an event raised to get the status of the order? Let's explore this in the next module. I really hope that you are as excited as I am to proceed in this course.

Events

Introducing an Event

Welcome back to the course C# Advanced Language Features. I'm Filip Ekberg, and now we are going to talk in detail about events in C#. In the previous module, we used a delegate to communicate that the order processing had reached a certain state. The idea of signaling that something reached a certain state or that something happened could also be seen as publishing or raising an event. For example, you could look at a graphical user interface. This application has a button. And when you press it, you want to run some code. This application uses WPF to create its interface. But the same applies to, for example, WinForms, .NET MAUI, and Xamarin as well. It uses a delegate to allow you to reference a method that is executed when you press the button. The only part of the system that should be in charge of invoking that particular delegate is the button control itself. It would be quite odd if any code throughout the application were able to invoke the click delegate. You only want that delegate to be invoked when that actual button on the UI was pressed. One of the issues with the publicly exposed delegate that we had earlier is that anyone could invoke it just like a normal method on that class. In some cases, that might be what you are looking for. However, when it comes to events in C#, that wouldn't be desirable. This means we have to take a slightly different approach. We are going to introduce a new keyword called event, and all of the knowledge about delegates is definitely necessary to completely understand how the events operate. When you talk about events, there are two important parts of this pattern, the publisher and the subscriber. The publisher owns the event and makes sure that anyone that's interested in this event will be notified. The subscriber can be anyone that is interested in when this event occurs. Let's start by looking at how an event is created. The idea behind an event is that it follows a very basic pattern. It all starts with the event keyword. The event keyword is followed by a delegate. While you could technically use any delegate together with the event keyword, you should not. An event will never return a value. Therefore, the delegate that it is based on will always be void. It's recommended to follow the same pattern used in .NET and as suggested by the guidelines. This means the delegate that is used together with the event keyword should be based on the EventHandler delegate or the generic alternative. The EventHandler delegate defines two parameters, the sender and the event data. The event data is also known as the event arguments. The sender is of type object and refers to the instance of the publisher that raises this event. The second parameter should use the base type EventArgs. This class should contain the event data. This could refer to the data which caused the publisher to invoke the EventHandler. An event can be defined in any class like you'd specify a field. The EventHandler is really just a delegate.

And if you forget to add the event keyword, it will behave like you saw in the previous module. When you do add the event keyword, the compiler will know how to handle this and make sure that it's only allowed to be invoked from within the publisher. This means you could only call the Invoke method on this delegate from within this class. The event keyword also generates code to expose something called add and remove as opposed to the getters and setters you have on publicly exposed properties. This ensures that whoever is accessing this event from outside the class is only allowed to add or remove a delegate from the invocation list. We now have an event called OrderCreated. This is all that is required to create an event that anyone with a reference to an instance of this publisher can subscribe to. Let's add this event to the OrderProcessor, and the OrderProcessor is now known as the publisher of this event. It owns the OrderCreated event, and it will ensure that it is notifying the subscribers when appropriate. As you've probably gathered, there's still a couple of things that's necessary to add before this is completed. The event needs to be invoked at the appropriate time. And the subscribers, if any, need to subscribe to this event. Let's proceed to invoke this event. The pattern to follow here is to wrap the invocation in a protected virtual method. This is because it will allow subclasses to override this method. They can then act on this prior to notifying all its subscribers through the base class. If the event is called OrderCreated, it's good practice to call this method OnOrderCreated. This will indicate that this method will execute when the order has been created. You can now invoke the event. This is also known as raising the event. Since the OrderCreated event is really just a delegate, when you invoke this, it will sequentially invoke all the subscribers that we have method references to. Under the hood, this all builds on the delegates that you learned about in the previous module. When you invoke this, use the null conditional operator. This will ensure that it makes a local copy of the reference and then performs the null check on that locally stored reference, which it then uses to invoke it. This is to avoid a race condition. The first parameter is a reference to the publisher. This is the instance of the class that publishes it. This is also known as the sender. The second parameter is the event arguments, also known as the event data. We'll cover event data in the next clip. So for now, we can use EventArgs.Empty to indicate that we do not wish to pass any particular data in regards to this event. Because this method is protected and virtual, a derived class could now override this. It could be useful in cases where you want to modify or extend the event data before the event is being published. The subclass should always call the base class to ensure that the event is properly published. In fact, the derived subclass couldn't call Invoke on the event even if it's

so liked to. The only class that is allowed to invoke the event is the class that owns it. That's why you'd have to call `base.OnOrderCreated` to ensure that the event is being raised. Now that we have a way to invoke the event, it's a good time to call the method `OnOrderCreated` at an appropriate time. Let's assume that after the call to `Initialize`, the order has been formally created in the system. After the call to `Initialize`, we'll call our virtual method `OnOrderCreated`. If there's a subclass that overrides this, its method will be the one that's executed. This provides further extensibility to our system. Everything inside the `OrderProcessor` is now set up in accordance with the guidelines, we have an event that uses the `EventHandler` delegate which specifies the `Sender` and `EventArgs` parameters. An event is always going to be void. There should never be any value expected to be returned from an event. It's simply a publish and subscribe pattern. The publisher should never care about what the subscriber does with the event and its event data.

Subscribing to Events

For these situations where you'd want to get a result back, you would rely on your own delegates that we looked at in the previous module. That's not a scenario suitable for events. Here's another important thing to remember. `OnOrderCreated` is a normal method call. In its turn, it invokes the delegate `OrderCreated`, which happens to be declared using the event keyword. That means it can only be invoked from within this class. When the delegate is invoked, all the subscribers of this event will be sequentially executed one at a time, just like one method call at a time. This is important because if you have a subscriber that takes a lot of time processing the event, this would make the entire order processing slower. Just like if you had a slow method running inside the `OrderProcessor`, we can easily visualize how this works. Head over to the program. This is where we are going to set up our event handlers. Any class that has a reference to the `OrderProcessor` could attach an `EventHandler` to the events. I can't stress this enough. If a class acts as a subscriber and attaches an `EventHandler` to your publisher, it is extremely important that it also unsubscribes from those events when the class is no longer being used. We'll dig into this a little bit more later on in the module. But for now, please keep in mind that whenever you subscribe to an event, you should always be unsubscribing at an appropriate time. You'll see the publicly exposed events on the `OrderProcessor` instance. In Visual Studio, this is highlighted using the thunderbolt icon. Whenever you see this, you know that you have an event. This allows you to add or remove delegates from the invocation list. All

you have to do, just as we've explored plenty of times now, is to reference a method that corresponds with the signature. To visualize how the handles are invoked, I'm going to add three event handlers expressed using lambdas. The lambda is created by defining the required parameters, which is the sender and the arguments. We can construct a lambda that first simulates a bit of processing by using a `Thread.Sleep` for 1 second. Then print a number to the console. Let's do this three times. These anonymous methods correspond with the `EventHandler` delegate, and `EventHandler` is just what these methods are normally called. The program is now our subscriber and has three different handlers added to the `OrderProcessor`. Before you run the application, what do you think is going to happen? Will all these `EventHandlers` be executed simultaneously? Which one will be executed first? What happens to the rest of the execution in the `OrderProcessor`? Let's run the application to find out. After the order has been initialized and the event is raised by the publisher, this invokes the first handler, which takes 1 second to complete. When that is done, it proceeds to run the second one. After that is done, it runs the third. After all of the `EventHandlers` have completed their processing, the `OrderProcessor` proceeds to execute the rest of the code. This is simply because each `EventHandler` is executed sequentially in the order they were added. A publisher may have multiple different events defined. This is something you will see a lot, especially if you work in UI applications. With events, the publisher owns the invocation. And also, the events should be using a delegate that conforms with the guidelines. Another important aspect is that, in most applications, the associated `EventHandler` and the subscriber will be attached and subscribing to that event for a long period of time. In particular, you will see that more often in UI applications. Now that you've seen how to create an event and how it's different from a delegate, let's proceed to learn about event data and how that is passed into your event.

Using Event Data

The next step is to look at how data can be passed into an event handler. The `EventHandler` delegate lets you pass an instance of what is called `EventArgs` as a second parameter. You could inherit from the `EventArgs` class and create a data container for the specific event details that you want to communicate. If you do this, the subscriber would have to cast the `EventArgs` to an instance of the correct type. There's a much easier and better way to achieve this. Instead of using `EventHandler`, there is a generic equivalent. Prior to .NET 4, the generic `EventHandler` required that the generic type

used with the delegate inherited from EventArgs. This is why you will often see libraries and applications that use event handlers where the event data inherits from the EventArgs type. There's definitely a benefit to having a common base type as you could attach any event handler that matches the object and EventArgs signature to that event. While we won't inherit from EventArgs in our case, it is completely okay to do so. We are going to create a class that holds the event data for the OrderCreated event. A suitable name for this class is OrderCreatedEventArgs. This is an old naming convention that is still wise to follow. It gives a clear idea of what the purpose of this class is. The container of data could have a reference to that order that was just created. This would contain the current status and the values for the order. Another common approach, of course, depending on the purpose of the event is to have the old and the new value in the event data. Let's say that the order creation also modifies the price depending on the customer that purchases the items. Maybe the customer has a standing 20% discount. In that case, I'd like to recommend that you have a property that contains the old price and one that contains the new total price. This way, the subscriber of the event will know that there was a change in the price, and they could communicate that properly. Now that we have the class that represents our event data, we need to specify that the event handler will use this. This is where we are going to change the OrderCreated event handler into a generic version. Doing this requires a couple of changes. First, change the event to be an EventHandler of type OrderCreatedEventArgs. Next, you'll notice that invoking this delegate and raising the event is showing an error. This is because using EventArgs.Empty is no longer applicable. You will now have to pass an instance of the event data so that the subscribers to that event knows what happened. We'll ask for this data to be passed into the virtual method so that subclasses don't have to worry about setting this up. Change the method signature of OnOrderCreated to accept the OrderCreatedEventArgs as a parameter. Of course, this requires us to pass an instance of the args into this method call. The compiler can infer the type for us, making the instantiation of this type a little bit cleaner. We can map the order into the Order property, and let's hard code the OldTotal and the NewTotal values to illustrate how this could work. Because we previously used lambdas to define the event handlers in the program, you don't have to change anything to run the application. Let's just clean it up a little bit and leave just one lambda statement that we'll set a breakpoint in. Running the application with the debugger attached shows that we reach the event handler in the subscriber. The sender is the OrderProcessor, and the variable args contains the event data. You can see the new

total and the old total and the order itself. Of course, anyone that subscribes to this event will now get the same details. As an example, let's say that you want to add logging to this event so that when an order is created, you have a specific method you want to run that logs the event. We can create an event handler that matches the delegate. This lets you log the event. This will then be executed in the order it was added. If you have a generic log handler that matches the non generic EventHandler delegate, this is where it could make sense to inherit from EventArgs on your custom events data class. This log method, for example, takes the sender and the EventArgs, meaning that this would match any event handler, either non generic or the generic event handler that have a type that inherits from EventArgs. This shows a good reason why sharing the same signature and base type is a good idea. I'd like to make another change to the OrderProcessor. Instead of the method Process accepting an action that is executed when the order has been processed, I'd like this to be an event instead. One benefit of that is that we could have multiple event handlers executed when the order has completed processing. To do this, we'll introduce a new event called OrderProcessCompleted. This will use the generic EventHandler, and we'll create a class that represents this event data. I'd say a good name would then be OrderProcessCompletedEventArgs. The event data will only contain the order itself. Do you remember where you should invoke the delegate? That's right, in a new protected virtual method. This will be very similar to the method we created for invoking the other event. Give the method an appropriate name, take the event data as an argument, and invoke the delegate. Don't forget the null conditional operator. Let's call this method at the end of the Process method. We could now subscribe to this and introduce an event handler. This could be to construct a shipping label for the order as soon as it's completed processing. Let's take a look at another example of using events. This warehouse management application will have a button to indicate that the order should be processed. Don't worry about the specific syntax to create the button. This is SAML and a way to construct the user interface. I can create a button element and then explore its different events. The button is the publisher, and the window that we have in front of us is going to be the subscriber. I can create an event handler for the click event, meaning that when the button indicates that it's been pressed, the event handler inside the window will be invoked. This builds on everything that we've discussed so far throughout this course. If we run the application with the debugger attached, you will see that when the button is clicked, this event handler is invoked. We won't dig into exactly what the EventArgs mean for this click event as it's very WPF-specific. The main window class that we are

currently in is also a publisher of events. For example, we can add an event handler for when a key has been pressed on the keyboard. If we do so and set a breakpoint in that event handler and press a button on the keyboard when the application is running, we can see that the EventArgs contains information about which key was pressed, if it was repeatedly pressed, and a lot of other useful information. Since you know exactly how these event handlers are executed, you might also understand that if you perform time-consuming operations in these handlers, the UI would suffer. Because the publisher is an element on the UI, the event handlers also execute on the same thread as those elements. This means you should be rather careful not to perform too long operations in these event handlers. You've successfully introduced a way to pass useful data into the event handler. You also saw how this is being used in, for example, a Windows application. With this Windows application introduced as an example, we are going to jump into the next topic and see why we might experience some odd issues if we forget to unsubscribe from our events.

Why Unsubscribing Is Important

It's best to unsubscribe from the events when the subscriber is no longer needed. Let's take a close look at a situation where forgetting to unsubscribe to an event can be a little bit problematic. In the Windows application that we previously used as an example, I am going to introduce the OrderProcessor. When the user of the application presses the button to process the order, this will start the order processing, as well as showing a window that will in its turn listen for when the OrderProcessor completed. Basically, the main window and a receipt window will share a reference to the OrderProcessor. The receipt window will subscribe to the OrderCreated event. When that event occurs, it can act on that information. Each time you press Process order in the main window, it will create a new instance of the receipt window and pass the OrderProcessor. Let's look at how to express this in code. First, add the OrderProcessor as a property on the main window and create an instance in the constructor. Again, this isn't a course on WPF or how to best build UI applications, but this will simply and fairly easily illustrate a problem that you can run into when working with events. The receipt window will accept the OrderProcessor, which we can store in a private field. We want this window to subscribe to the OrderCreated event so that it knows when to display the order receipt. Let's subscribe to this event using a lambda. To keep it simple, we can display a message box that we received this event. It looks fairly simple, doesn't it? Now, we can head back to the button click event

handler that is supposed to create an instance of this window. We are going to create the instance, pass the OrderProcessor, and then show the window. After that, we can call Process on the OrderProcessor. And so far, it doesn't look very odd or wrong, does it? What happens if we run the application and click the button? It seems like it's working as expected. Pressing the button Process order does, in fact, open another window, and it does show a message that the order is created. If you close the receipt window and press the button again, what happens? Ouch. That didn't work completely as expected. Did you notice that we received two message boxes? If you repeat this process again, close the receipt window, and press the button, there will be three message boxes. This is definitely an issue. Do you know what happened here? The event handler for the OrderCreated event was invoked three times. Why is that? We completely closed the receipt window, we no longer have a reference to that window, but somehow the delegates are still executed. This is exactly why it's important to unsubscribe from your events. The garbage collector is unable to clean up the receipt window because there is a delegate associated with it. The anonymous function created using the lambda is still associated with the invocation list on the EventHandler delegate. How would we unsubscribe in our situation? As I've previously mentioned, there's no clean way to do this. This is one of the issues with using an anonymous function as an event handler. This won't always be an issue. It really boils down to the fact that the OrderProcessor is a shared instance. It's not too uncommon of an issue, so how do we best fix this? In this case, it's better to not use the lambda, but instead create an event handler as a separate method. It's an easy change. All we do is introduce a method called Processor_OrderCreated and move the code from the lambda into this method. We still need to unsubscribe this method from the event. You want to do this when the instance is no longer relevant. In this case of this window, we could do this when the window is closing. There's a method that we can override called OnClosing, which is, in fact, also in its turn calling an event, but that's beside the point. In this method, unsubscribe the method from the event before calling the base OnClosing. If you rerun the application with the same test as before, this will now yield a much better result. Each instance of the receipt window will be cleaned up as it's closed, and there is no longer an event handler leak. The message box should show only once each time you click Process order. Hopefully, this is convincing enough to show you why it's very wise to unsubscribe from the events. I want to highlight that this is not only a problem in UI applications. This can happen in any type of application and could cause unexpected problems. The sample application is a simple way to show a way to reproduce this issue.

Don't be surprised if you run into this in the real world. I definitely learned about this the hard way myself. In my case, I had an application. Accidentally, an event handler was not unsubscribed when one of the views were closed, much like in our sample. The view that handled the purchase of an item wasn't properly disposed, and it didn't unsubscribe from its events. When a signal was sent that a purchase was completed, this resulted in an extra receipt being printed for each purchase that had occurred. Imagine my surprise when I tested this. Luckily for me, the issue was promptly discovered. And with a good understanding of events, it was very easy to solve. Let's look at this from a different angle. Take this class as an example. The whole idea behind this class is to handle some sort of logging. When you create an instance of this class, you inject the `OrderProcessor`. At a first glance, this may not seem odd. The class then registers itself as a subscriber on the publisher, which is the `OrderProcessor`. This means that the `OrderProcessor`'s event that you subscribe to has a reference to this class and its referenced method. Unless the class implements a pattern to unsubscribe its methods from the publisher, this is a bad idea. If you create an instance of that class and inject the `OrderProcessor`, then immediately after that, you set the reference to null. The event handler would still be invoked. This is because the publisher still has a reference to the instance. That instance cannot be cleaned up by the garbage collector until that reference is removed. The only way to remove that reference is to unsubscribe from that event. Exactly how you proceed to do that in this scenario can vary. This is where it's important to consider the overall architecture of the application. You want to do your best to avoid this type of design. In this particular instance of the logger, an alternative approach could have been to remove the `OrderProcessor` from the constructor and instead register an event handler at the application level, which will then call the method `log` on this logger. You could then make sure that you unsubscribe the event handler at an appropriate time, for example when a view closes. Another approach is to implement `IDisposable`, but we'll get to that in the last module of this course. The most important takeaway from this is that you should always unsubscribe your event handlers. Plan your application architecture so that it's easy to do so.

Summary

This module introduces you to events, how to properly set them up, and that it's extremely important that you always unsubscribe the event handlers. Before completely understanding how events work, you need to have a good foundational understanding of delegates. While events and delegates look

and act the same, one important difference is who owns the invocation. When it comes to a delegate, anyone that has a reference to the delegate can invoke it. With events, this is not the case. Only the publisher, which is the class that owns the event, can be the one invoking it. Not even its subclasses could invoke the event. We started off by introducing the keyword event. This is used together with a delegate. While you can use the event keyword with any delegate, you should conform to the guidelines and use the EventHandler delegate or the generic EventHandler delegate. When you introduce the event keyword with your delegate, you are now limited to only adding and subtracting method references from the invocation list. To invoke the event handlers, you must do this from within the publisher. This will let all its subscribers know that the event has occurred. Invoking the event handler should be done through a protected virtual method. This pattern is good to follow because it allows subclasses to extend the functionality of what happens prior to an event being raised. If a subclass does override this behavior, they must call the base class to get that event raised. After introducing the event and invoking it, we discussed how event data can be passed into the event handlers. We saw that one benefit of using a class that inherits from EventArgs is that the event handlers that only corresponds with the non generic delegate could still subscribe to those events since they would share the same base class for the event data parameter. You don't have to inherit from EventArgs, but it's good to know why it is useful. Working with events is extremely common, especially in user interface applications. It doesn't matter which type of UI application or what platform, I can guarantee that you will encounter events. Using a UI application, we saw an example of how to run into a little bit of an issue when we forgot to unsubscribe the event handler. While this isn't only a problem for UI applications, it's an easy way to explain and illustrate this potential problem. You must find an appropriate place in your subscriber where to unsubscribe from events. For UI applications, this is commonly done when the view disappears. And then, you can resubscribe when the view appears again. This issue isn't only applicable to events. This relates to all types of delegates. This is also a reason why using lambdas as event handlers is not ideal because there's no simple way to unsubscribe that lambda. Events and delegates are powerful language features, and I bet that you will most definitely use this throughout your work in C#. We've been talking quite a bit about extensibility. And next, we will focus on reusability. More specifically, what if we'd like to extend a class with additional methods, but we don't own that implementation and we don't really want to subclass it? After it's extended, we'd like to share this extension so that others can also use our additional

functionality. This is where extension methods are truly powerful. I hope that you have enjoyed the course so far. Continue to the next part where we are going to look at overloading and extension methods.

Overloading and Extension Methods

Method Overloading

Welcome back to the course C# Advanced Language Features. I'm Filip Ekberg, and now it's time to talk about overloading and extension methods. We'll start by looking at a recap of method overloading before we take a peek at what operator overloading is and where that could be useful. Then, we'll jump over and talk about adding methods without access to the class using something known as extension methods. You shouldn't confuse overloading with overriding. A method overload happens when two methods have the same name, but with different parameters. As an example, if we were to extend the class `OrderProcessor` with an additional method called `Process`, let's say that it takes two parameters. There would now be two methods available in the class with the same name, one that simply processes the order and the other allows you to pass a discount together with the order. They could also have entirely different parameters. They don't have to share any similarities at all. When calling the method `Process` on the instance of this class, the method that corresponds with the parameters that you pass to the method, that's the one that will be executed. This is determined during compile time. As an interesting example, if one method accepts an object and the other accepts the type `Order`, which one do you think is executed in this scenario? The fact is that it will execute the method that accepts an `Order` unless you would have casted the instance to an object before calling the method. If you haven't experimented with method overloading before, I strongly suggest that you take some time and add a few methods that share the same name with similar method signatures. It's very common that both libraries and the code that you write yourself exposes methods that accept different parameters but share the same intent. Understanding overloading before jumping into extension methods is a good idea. We'll cover extension methods thoroughly in a little bit. But before

we do that, there's one additional type of overloading that I'd like to take a look at, that is operator overloading.

Operator Overloading

C# has a lot of operators. Some of them can be overloaded to change the behavior of how they work. Let's look at an example from .NET where this is applied. I've constructed two instances of the struct `DateTime`. You can learn how to properly work with dates and times in .NET in one of my other courses. These aren't numbers. In fact, it's two references to instances that represent a date and time. So what would subtracting one date from the other mean? Let's say that we want to figure out how many days there are between two dates. If we subtract, this will result in a completely different type. This is a time span, which will give you the total days, minutes, seconds, and so forth that have elapsed between these two dates. This is a great example of where overloading an operator makes total sense. The `DateTime` struct introduces a lot of operator overloads, one of them for subtraction. This takes two instances of `DateTime` and returns a `TimeSpan`. Introducing an operator overload may seem simple at a first glance. But sometimes, it can be rather tricky. For comparison operators, such as equals, you're required to also implement not equals. This is true for less than and greater than as well. If you ever have to introduce an operator overload, it's a great idea to consult the Microsoft docs to ensure that the specific operator is overloaded in accordance to their recommendations.

Implementing an operator overload requires you to follow this pattern. You need to declare a public static method followed by what this will return. The return type can vary. It doesn't have to be the same type as the class that you are adding it to. And the type is sometimes dictated by which operator that you are overloading. You then use the operator keyword followed by the operator. The final part of the method signature for this overload are the parameters. This is also dictated by the specific operator that you are overloading. In our example, let's say that you want to allow two orders to be combined. To make it easy, you want to overload the plus operator. That would enable you to add one order to the next and have an order returned. In most situations, a better approach would have been to introduce a method called `Combine` that do exactly this. Introducing an operator overload could introduce unnecessary complexity in your code base. I'd recommend that you really think more than once before introducing an operator overload. In situations where it feels natural to perform addition, subtraction, or any of the other operators such as in the case of `DateTime`, that would be a viable case

for introducing an operator overload. If you want value-based equality, that is where you'd compare the value of the properties instead of the reference, I'd recommend that you resist the urge to overload the equality operator yourself. And instead, wait until later in the course where you will learn about record types. If you really can't wait and you want to introduce value-based equality, you can generate this with the help of Visual Studio. Simply go to the class declaration, click on the tooltip, and choose to generate equals and get hash code. This lets you select the properties of the class that you want to include in this value-based equality. Here's also an option to implement `IEquatable of T`, as well as the operators. If you check both of these checkboxes and hit OK, this will generate some code for you. First, it will add the interface implementation to the class for `IEquatable`. This introduces the public method `equals` that returns a `Boolean`, which lets you compare the current instance with another instance. As you can see in the method body, it compares the properties with each other to validate that the values of both the instances are the same. It then adds the override for the `equals` method that accepts an object. This is then passed to the other `equals` method with this object cast to an order type. It also overrides the method `GetHashCode`. Doing all of this yourself manually can be tricky, so use the built-in tooling to help you generate these methods. Finally, it also introduced two operator overloads, one for `equals` and one for `not equals`. This will let you compare two separate instances of orders, and it would then compare the properties with each other. You can pause the video and try this out. Run the application with the debugger attached and see how this works. C# also provides the capability of overloading something known as a conversion operator. If you introduce a type, such as the order that we have been working with, if you want to cast this to, for example, a decimal, the compiler would tell you that this doesn't work. That makes sense because what would converting this into a numeric representation entail? There is a way to define this, and that is done with the conversion operator. There are two types of conversions, implicit conversion and explicit conversion. An implicit conversion is, for example, when the compiler can implicitly determine the type that you want. A good example of this is a method call or a variable declaration. Performing this conversion should be safe. Hence, it shouldn't throw any exceptions, and it shouldn't lose any data. On the other hand, you can require that the type is cast explicitly. This will then also mean that you do not guarantee that it will always work and that data can be lost. This is why there are two additional keywords, `implicit` and `explicit`, that are both used together with the operator overload that we've been looking at. The only difference between using these two keywords is what we've just talked about. One

will be used for implicit conversions. And you guessed it, the other one is used where you perform an explicit conversion. Instead of the operator, you declare the type that you want to convert to. And the parameter is the type that you convert from. You could introduce a conversion that allows for the order to be cast to a decimal to get the order total. This may make sense, but it doesn't make sense to do the other way around. It wouldn't make sense to convert a decimal into an order. You do not have enough data to convert that back to the order. Although in a lot of cases where you do overload the conversion operators, it also makes a lot of sense to do it both ways. As an interesting side effect of introducing implicit conversion is that if you pass an instance of an object into a method such as `Console.WriteLine`, it may use the implicit conversion to get a representation instead of using `ToString`. Many developers will go through their entire career without having to write any operator overloading at all. If you end up in a situation where you have to introduce it, maintain, or change an operator overload, you now have an idea of what that entails. And it's a really good idea to consult the documentation for each of the operators that you wish to overload. With a brief understanding of operator overloading, let's now switch gears. Next, we'll be looking at how you can extend a type with a method without having access to the class. It's time to introduce an extension method.

Introducing Extension Methods

You don't always own the code base that you want to build extended functionality for. In some cases, you're not even permitted to introduce a subclass. Or simply, it's just not a desirable solution. Extension methods, as you will see, are not limited to classes. I'd like you to imagine adding a method to all types that implement a specific interface. This is a truly powerful way to create a reusable piece of code that can be shared across all your projects that use those types. Of course, there are multiple ways to build helper classes, but using something called an extension method allows you to create a method in a completely separate library that acts like an instance method on the given type. It can be useful to provide completely new functionality or act as a method overload by accepting different parameters than what the method on the existing type does. You should only resolve to this when you do not have the capability of extending the original class. If you are building a library of helpers that extend functionality on a given class, sharing this within the business or on NuGet can prove really helpful. Extending a type is fairly simple. You need to follow a basic pattern. First, you need a static class that holds all the extensions for a given type. It's a good idea to create one class per type that

you want to extend with additional methods. Let's say that you want to extend the order. You'd create a class, which you'd call `OrderExtensions`. This class can now contain static methods that have the sole purpose of acting like an instance method on the order. The method signature for an extension method is slightly different from what you are used to. You define the return type and the name for the method just like you normally do. Where this will start to look a little bit different is when it comes to the parameters. The first parameter will be the type that you are extending with additional functionality. To define that this is an extension method for the order type, you use the keyword `this`, followed by `order` and the name for this parameter. What this now means is that this method could be used like an instance method on the order as long as you include this library and use the namespace. The compiler will take care of passing the actual instance as the first parameter to this method. Really, this is just like introducing a helper method, but it acts like it lives on the order itself. This is also why you'll only be able to access properties and methods on the type that you are extending, which are public. You could add additional parameters as well. This would mean that when you invoke this method on an order instance, you have to pass that parameter just like a normal method call. Of course, we own the implementation of the `Order` class, so creating an extension library for this wouldn't make much sense. It would be better if this functionality lived somewhere else. You may be wondering what happens if a method already exists on the type that matches the exact signature that you are creating using one of these extension methods. The compiler will make sure that the extension method would never be used if the type has a matching method signature. This means extension methods have the lowest possible priority. Since the extension method is a simple static method, you could, of course, call it directly, which also explains why you're not able to access private fields, properties, or methods on the type because, internally, this is exactly what's going to happen.

Creating an Extension Method for `IEnumerable<T>`

I'd like you to consider the `IEnumerable` interface. This represents a type that you can enumerate. Think of a collection or list of data that you'd like to use with a `foreach` loop. Before iterating through the data, you first want to find certain items. How would we go about building a reusable method that allows you to search for items in a collection? Since we do not own the `IEnumerable` interface, we can't add methods to that. Therefore, it's appropriate to do this with extension methods. We start off with the class. This is a static class that represents what type we are extending. The class

IEnumerableExtensions will now contain all the methods that we have added to the IEnumerable. I'd like to introduce a method called `find` that in its turn returns a list of items that match a given search. Because we want this to work with any type, we are going to rely on generics. This allows you to build functionality for this without knowing the exact type. The method is a public static method, which returns an IEnumerable of T. We'll call this method `Find`, and we also have to specify that this is a generic method. It means that the type can be defined by the consumer of this method. Because this is an extension on the IEnumerable, the first parameter will be the instance of the IEnumerable of T. Let's call this source. We'll also accept a second parameter. This will require that you understand delegates because we are going to rely on the `func`. This second parameter will accept a delegate that determines if it's an item that we are looking for. This means the delegate will accept an instance of T passed into it. And after it's performed its comparison, it will return a Boolean to indicate if this was the item that we were looking for. We can then use a `foreach` loop. And in the body of the `foreach`, we can call the `func` and pass the current element. If there is a match, we can use the `yield` keyword to send that back to whoever is iterating through this enumerable. The `yield` keyword is a contextual keyword, and using this makes it easier to create an iterator. An alternative approach would have been to first create a temporary instance of a list and add elements to this one by one before returning the entire collection. That would have behaved a little bit differently. As with the `yield` keyword, each item is returned one by one instead of a chunk of elements. Whoever now adds a reference to this library and includes the namespace that this IEnumerable extensions class lives in will have the capability of calling `Find`. We could have a list of items that we want to search in. And as you can see here, Visual Studio does indicate that this here is an extension method. You can then define a lambda expression to indicate what you are looking for. Each iteration inside this extension method will then call this anonymous function to evaluate if it's an item that we are looking for. This is what makes extension methods extremely powerful. If we'd like to, we could now package this as a NuGet package and have other developers inside or outside of our business use this anywhere they have an IEnumerable of T. If you do not own the code base that you want to build extensions for, this is a great way to build a reusable set of common methods.

Creating an Extension Method Library

You can create extension methods for everything, for example classes and records, as well as for structs and interfaces or even delegates. As mentioned, you should only create extension methods for types that are out of your control, meaning that you do not own the code base or are unable to change the implementation. I'd like you to imagine that we are working in a really large organization where the domain classes are shared among many different projects. You install the domain library through the private NuGet server. You know that at least a couple of more projects depend on some functionality that you are going to build, which extends the Order class. This is a great opportunity to build a reusable extension library. The Order class in the solution doesn't have much functionality attitude. It's a simple representation of the order data. For simplicity, we are going to keep the domain library in the solution. You just have to imagine that this is installed through NuGet and that you do not have access to that source code and really outside of your control. I cannot stress this enough. As we talked about earlier in this module, it's good practice to keep your static classes focused on a single type. If we want to extend the order, you'd create an OrderExtensions class. If we also want to extend the shipping provider, you guessed it. You'd create a ShippingProviderExtensions class. The shared functionality that we want to build is a way to retrieve a report for an instance of an order. This will return a string when the generation is completed. The method GenerateReport takes the type it's extending as its first parameter. Remember, this could have been any reference type, value type, interface, or even delegate. The three things that make this an extension method are it lives in a static class, it's a static method, the first parameter uses the this keyword. This method will now construct a report based on the order. We can use string interpolation to easily construct this string. Let's also make an overload that takes a second parameter. This illustrates how you can introduce method overloading for your extension methods. And now, we can call GenerateReport. Add a using statement for the WarehouseManagementSystem.Domain.Extensions. You will see that you have both the different overloads for GenerateReport. If you run this with the debugger attached, you can step into the extension method just like you would expect. What do you think happens if the method GenerateReport exists on the Order class? We can go ahead and try that. Add a method that matches the second overload that accepts a string, but use a different parameter name. After this is added, let's go back to the program. It will now look like there are three overloads for this method, although two of them are identical as both accept a string as the parameter. This means if you run the application, you would have a different result compared to before as now we have this method on the Order class. The

compiler will make sure that it always uses the method available on the actual type before using an extension method, no matter if it was included using a global or normal using statement. Because we have different parameter naming, we could use a named argument to specify exactly which of these two methods we intend to run. If you change the call to the GenerateReport method and include a named parameter, you can make sure that it runs the method from that extension method library. It's worth noting that we didn't get any warnings or compiler errors when the order had the same method added to it. The application just behaved a little bit differently when executed. You've now introduced a library for extension methods. While this course does not cover how to package this as a NuGet package, it's worth to keep in mind that this could be a great idea if you are building reusable code for types that are commonly used within your company. Next, we are going to look at a very common library of extension methods that I can guarantee that you are going to be working with quite a lot.

Real-world Implementations of Extension Methods

Earlier in this module, we introduced a rather powerful extension method to the IEnumerable. I can think of plenty of additional extension methods that would make my life easier when working with collections of data. Luckily, someone else thought of this before. You may have seen this, but the fact is that .NET comes with a great set of extension methods. A bunch of these extension methods are built for types such as IEnumerable and IQueryable. The most used extension methods by far are what is known as LINQ. LINQ is short for language integrated query. There are other courses in the Pluralsight library that covers how to work with LINQ in great detail. Although since this library is built using extension methods, I think it's a great example of what you can accomplish if you build and distribute powerful and reusable extension methods. If you construct a list, array, or any other type of collection that implements the interface IEnumerable, you will be able to use the extension methods from the System.Linq namespace. Our program lives in a C# project that has ImplicitUsings enabled. That is why you do not manually need to include the using statement for System.Linq. If you try to use these methods in your own application and you simply cannot find them on your instance, make sure that you have the using statement at the top of the file or included using a global using statement. With the extension methods available from System.Linq, you no longer need the custom extension method that we introduced earlier because LINQ introduces that filtering capability, as well as a whole lot of additional methods. Just to explore a few of these different methods, you could use an extension

method to retrieve the average on a collection of elements, let's say the average price of all the items in your order. This takes a func. Another extremely useful method that I personally use a lot is an extension method called `Where`. This lets you filter the collection or, in other words, search for all the items based on the predicate. Again, you'll most often see this being used with a lambda. There are, of course, a whole lot more extension methods, and some of them are for more specific and complex situations. For example, how do you determine which elements are shared between two collections? There's an extension method that does this for you. It's called `Intersect`. You can call `Intersect` on your collection and pass the second collection to find which elements exist in both of these collections. Or of course, you could build all of this functionality yourself or have helper methods construct it in other ways. Using extension methods for the type of functionality and interactions that we have explored is a great approach. As a final note on reusability and readability, I'd like you to take a look at this fluent chain of method calls. First, we filter through the enumerable based on the criteria defined in the anonymous function. Then the order result is ordered based on the price. And finally, we want to select the five cheapest items. These are all extension methods, which are all powerful on their own. But since they follow the pattern of returning an `IEnumerable`, you can easily construct a chain and add additional method calls to end up with a desired result. Again, if you want to learn a little bit more in depth about LINQ, there are plenty of other courses in the library. This has shown you a great usage of extension methods and how powerful that can be.

Summary

This module started with a recap of what method overloading is and how that works. Method overloading is when two methods share the same name but have different parameters. We saw a few examples of this, both when the `Process` method had an alternative overload, as well as an extension method that provided a different overload. We then looked at a different type of overloading. You were introduced to the concept of operator overloading. C# contains a large amount of operators, and a lot of these can be overloaded. It's worth noting that many developers will go through their entire career without having to introduce operator overloading. Overloading an operator is done with a static method. The return type may be dictated by which operator you are overloading. You then introduce the operator keyword followed by the operator. Finally, its required parameters. If you work in a solution where operator overloading is appropriate, it's wise to consult the Microsoft docs to ensure

that the operator is overloaded accordingly. Understanding operator overloading makes you appreciate the types that cleverly implement this. We saw an example with a datetime where you could take two instances and subtract one from the other, and this would result in a completely different type, which represented the time span between these two dates. If you build complex mathematical types, introducing operator overloading is definitely going to be handy. You then saw how to use Visual Studio to generate equals, get hash code, and the equality operators. Then, we saw an example of the conversion operators. You learned that you can introduce an implicit or explicit conversion. This means you can introduce a method that lets you convert an instance of your type into anything else as long as it's an appropriate conversion. If your conversion could lose data or throw an exception, you should implement the conversion using the explicit keyword. Finally, we looked at extension methods. If you want to add methods to an existing type, but you do not own that code or not allowed to modify it, using an extension method lets you add a method to a type just like if it was an instance method. This is a great way to create reusable code for common functionality on a given type. We saw this being used on an IEnumerable, but also on the order. Obviously, the order is a part of our solution, so you have to imagine that it's not. This could be coming from a shared domain library that was retrieved through the company NuGet server. An extension method lives in a static class, and the method signature need to follow a very basic pattern. It needs to be a static method, and the first parameter uses the this keyword to indicate the type it's adding a method to. While the static class could contain extensions for everything, it's wise to group your extension methods into classes that represent the type that you are extending. This will make your code more readable and maintainable. These methods will show up as instance methods on the type itself. You could call these methods explicitly by simply using the class name dot the method and pass the parameters just like any normal static method call. It's important to remember that the compiler will see the extension methods as the lowest possible priority. This means if there's a method on the actual type that has the same signature as the extension method, it would never call the extension method. This may happen in situations where you previously extended a type with functionality, and then someone updated the type with a similar method signature. We looked at what is probably one of the most commonly used extension method libraries out there. This is System.Linq. It provides an extreme amount of extensions to IEnumerable and other types as well. This will let you work with collections of data in a much more pleasant manner. There's a whole lot more to LINQ than what we have looked at in this course. It's a

powerful set of extension methods, and it shows exactly what this language feature was built for. With collections in mind, I'd like you to think about the following. When you filter through a collection or only want to select a subset of the properties available on the type, it would be really handy if there was a way to create a tiny representation of only the data that you want to use. Creating a class that will only represent the order number and sum, for example, may seem excessive, especially if this container of data will only be used inside the current method. In the next module, we'll be looking at what's known as an anonymous type.

Anonymous Types

Introducing Anonymous Types

Welcome back to the course C# Advanced Language Features. I'm Filip Ekberg, and now it's time to look at a convenient way of defining a type to group a subset of properties or values. This language feature is known as anonymous types. Another way of looking at the anonymous type is that it's a temporary class used in the current method. The intent is never to return the anonymous type outside of the current method. There are ways to work around that, but we'll discuss the drawbacks of this later in the module. There are some good use cases for anonymous types. For example, say that you have a collection of data where you first want to apply a filter, then order, and finally get a new less complex representation with only a subset of the properties. This data is then automatically populated into a data grid in a UI application. You could create a new class for this, but that seems a little bit unnecessary, especially when there's a language feature that will make this easy for us. Let's look at how to create an anonymous type. You create an anonymous type using the `new` keyword followed by the object initialization. This looks like if you are constructing an instance of a type with object initialization, but you forgot to put the type of the object that you are instantiating. When the compiler sees this, it will, in fact, generate a type for you. If you were to decompile this and look at the generated code using something like IL Spy, which is a program that lets you look at the code that was generated by the C# compiler, you can see that the compiler generates a completely new class. The class is generated using a special name. You can see that the anonymous type that is generated

overrides a few methods. For example, it overrides equals. Let's get back to that in just a moment. An anonymous type without any properties and values doesn't make much sense. To add some properties to the type, you simply use it like an object initialization. This is by first specifying the property name and then assigning its value. The compiler will figure out the type of the property by looking at what you assign to it. If you had two properties which were both set to integers, the generated class will have a strongly typed property. Since C# is a strongly typed programming language, it makes sense that it generates this class. Looking at the decompiled code again, this will now have a little bit more code generated. Before looking at the body of the class that was generated, notice that the name of the class was also changed into something that represents what this anonymous type contains. The generated class has two backing fields for the properties that we declared. These are going to be read-only, meaning they can only be set when the anonymous type is instantiated. If you want to change the values, you cannot do so without creating a new instance. Looking further at what was generated, the override of equals performs a value-based equality. If you create two instances that both match the same signature used by an anonymous type, the compiler will be smart enough to use the same type for both instances. When the method equals is called, it will then perform an equality check on all the properties. Notice how it does not overload the equality operators. This means if you perform a normal equality check, this would yield false, while a call to equals would be true. This is because the equality operators check the reference equality, while equals look at the properties. The compiler is also smart enough to figure out the appropriate property names. You don't have to explicitly set the property names. You'll see that this is truly useful, especially when used together with LINQ. Not having to repeat yourself. And instead having the compiler do a little bit of work for you is great. An example of this could be if you construct a new anonymous type that contains only the order number and the total price. You could then access the properties on the anonymous type through the instance. As you can see, the compiler infers the names for the properties. It uses the same casing as to property or field that you are inferring the name from. This could, of course, be changed by explicitly declaring the name of the property on the anonymous type. It won't always be possible to infer the name of the property. If you call a method to retrieve a value that you want to store in one of the properties on the anonymous type, how would the compiler know which name to use for the property? Should it use the method name? Well, maybe that would have been a great idea. However, that's not the case. For method calls, you must explicitly

declare the name for the property. It's completely fine to have some names inferred and some of them explicitly defined. For these situations where the compiler can infer the names, you could skip the explicit property names. The properties on the anonymous type can be of any type just like you'd use with a normal class or struct. The anonymous type is only meant for properties. You cannot have methods on them. But as you've seen earlier in this course, you can use a delegate as any other reference type. This means if the anonymous type would have an action, func, or a delegate property, this would, of course, be possible to invoke like a method. But this isn't something that you'd normally want to do. The purpose of the anonymous type is to allow you to group some data in a local class without having to go through the trouble of creating a completely new class. It also gives you the benefit of being read-only. Now that you've learned what an anonymous type is and how it's constructed, let's look at a common use case for anonymous types, namely when working with collections of data using LINQ.

Using Anonymous Types with LINQ

The most common use case for anonymous types is most definitely when using LINQ. We briefly looked at using the LINQ in the previous module that covered extension methods. You can use LINQ in two different ways. Either you use what we looked at earlier, which is known as the method syntax. You can alternatively use a query syntax that would allow you to express complex queries in a more simple manner. Whichever of the two you prefer, they can be used together with the anonymous type. In the warehouse management system, let's say that we construct a list of orders. Then, each order in its turn has a list of items. We are going to introduce an overload for Process on the OrderProcessor that accepts an IEnumerable of Order. In this method, I'd like for you to create a query that produces an order summary. To do this with the LINQ method syntax, we call the extension method from System.Linq called Select. This is a simple way of iterating each item and then retrieving that. The call to Select accepts a Func of Order and TResult. This is a generic delegate that accepts the order as the first parameter, and then you can return anything you'd like. This means we could get a complex type passed into this anonymous function and construct a new simpler representation. This is a great place to introduce an anonymous type. For each order in the list of orders, we are going to select a new representation and create that using an anonymous type. The summary will contain a property that we call order. This is pointing to the OrderNumber. It will also contain the amount of items. To

count all the items in the list, there's an extension method called Count. And finally, the total value of the order based on summarizing all the prices on the line items. And to get the total value of this order, you can use a method called Sum. This method accepts a Func of Item and returns a decimal. Therefore, you can use this to get a summary of all the prices. If we go ahead and call this method in the program and then run this application with the debugger attached, as we step into this method, we can see the result of our LINQ query. We can see that the result of calling Select produced an IEnumerable containing an anonymous type. Each item in this array contains an instance of the anonymous type. As long as we use this within the same method, we'd be able to reference the properties created on this anonymous type. The compiler will have generated a class for you that corresponds with the anonymous type used in the Select method. We can verify this by looking at the decompiled source code and see that it did, in fact, produce an anonymous type with the properties Order, Items, and Total. To take this one step further, since this produced an IEnumerable, you could use LINQ on the result from the Select method as well. Let's order these summaries by the total value of the order. To do this, we can use a method called OrderBy on the summaries instance. This will behave very much like the other LINQ methods that we've looked at. In this case, it's going to be an anonymous type that was produced by the Select method. If you hover over the summary variable used in this lambda, you can see that this is the anonymous type. The compiler inferred this for you. Since this is a generic method and the language is strongly typed, you can access all the variables on this anonymous type, just like if the class was created by yourself. We will now have a list of order summaries that are in order of the total price. You can iterate through this enumerable, use the properties from the anonymous type, and print a summary to the console. This works really great. As long as we use the anonymous type within the same method, we won't have any problems. When you work in UI applications where some of the components in the UI are data-bound, that means you want to attach what that element in the interface represents by binding it to an instance. Using an anonymous type can be very powerful. Let's jump over to the Windows project and look at an example of how to do this. I'm going to extend the UI with a data grid. Again, this course isn't about WPF, but it's going to illustrate a good use case for anonymous types. With this data grid available in the application, I'd like to populate this with the summary of my orders. I'm going to head over to the MainWindow.xaml.cs and introduce a new method that works with the order summaries. I'll create the same list of orders and items that we had in the program. Imagine that this data was loaded from a

database. I don't really want to make it too complex, so let's just create a temporary list in this class. Now, introduce the same functionality that we looked at earlier. This will select an anonymous type based on each element in an array of orders. After this new array of anonymous types have been produced, we can order this using LINQ. This should look very much like what we had earlier in the console application. I'll use some WPF-specific code to bind this collection as a source on the data grid. What the data grid will do now is produce a nice representation using the property names on that anonymous type. And then, it will fill this grid with all the data from our array. If I call this method when the window is created and run the application, you can see that the data grid is filled with the order summaries. It automatically created the columns and headers based on the anonymous type. You can add an additional property to the anonymous type to, for example, see if the order is ready for shipment. Then, if we rerun the application, this will now have updated the data grid automatically based on the anonymous type. This is where anonymous types are truly powerful. So far, we've seen how to construct an anonymous type. We've talked about it being read-only and the fact that we shouldn't use it outside of the current method. There is a way to create a new instance of an anonymous type based on the previous instance with some modified values. Let's look at this next.

Modifying and Returning Anonymous Types

As you've seen, anonymous types prove to be powerful when used together with a LINQ and when you want to present this data in the UI through data binding. While the anonymous types properties are read-only, there is a way to create a new instance with copies of all the properties available into another instance. Then, you can specify which properties you'd like to contain different values. Doing so is known as a non-destructive mutation. You're not mutating the original instance because you can't. It's read-only. Instead, we're going to introduce a new concept and a new keyword. With an instance of an anonymous type, you can declare that you want a copy of this with one or more properties changed. This is known as the `with` expression. It was originally introduced together with record types, which are covered later in this course. Using the `with` expression does not allow you to add or remove properties. It's simply a way to create a new instance of the same anonymous type with some different values. For example, let's say that you want to change one of the order summaries and apply a 25% tax on the price. Because it's read-only, you have to construct a new instance. We can take the first order summary and create a copy of this instance using the `with` expression. Inside this

statement, you can specify the properties that you want to change. Since this is a generated type and we're using it in the same method where it was generated, the compiler knows exactly which properties are available. And therefore, the IntelliSense will show us all the available properties that we can modify. Now, we can state that we want to change the total price of this order summary to apply a 25% tax on this single instance. We now have a new instance of this order summary. You could dereference what the first element in the array points to and have it pointing to this new element instead. The benefit of creating a new instance, instead of allowing to modify the original instance, is that in multi-threaded environments and applications mutating objects can cause problems. The `IEnumerable` doesn't allow you to replace an element, so you'd have to either retrieve this as a list or as an array to do so. It's worth to keep in mind that just like with any normal type, if you have a property that is a reference type, that simply points to the instance, and that is true for anonymous types as well. We can experiment with this by adding the line items to the anonymous type. What do you think happens when the anonymous type is copied using the `with` expression? For the property `line items`, it only copies the reference. This means it does not make a copy of the entire array. For strings and value types, there will be a copy. But for reference types, that's not the case. What does that mean? Add some code that retrieves the first line item from the first order summary. Then modify its name. Since the line item is a simple reference type that exposes a public property with a getter and a setter, we are allowed to do this. Before we run this, take a moment to think about what's going to happen. Will the order summary with the discount have a different name for the line item? Let's run this to find out. As you can see, the name was changed for both of the instances. This is because the enumerable for line items is a simple reference to a collection in memory. Both instances of the anonymous type will point to the same collection. This is something that you should definitely keep in mind. You've seen how powerful anonymous types can be, and it will reduce the amount of unnecessary code in your applications. The only thing that we haven't looked at is how to return an anonymous type from a method or how to pass it as a parameter. As mentioned many times now throughout this module, that is not the intent of anonymous types. However, that doesn't mean it's not possible. The anonymous type that is generated is a normal class. As other reference types, it ultimately inherits from `object`. A method that has the return type `object`, a method that accepts an `object`, or a property that is of type `object` could point to an anonymous type. You can change the method signature of the method process to return an `object`, then return the instance of the

anonymous type. If we then go back to the program, we can capture what is returned from this method. And we now have a local variable of type object, which is the instance of the anonymous type. This will make it difficult to work with. IntelliSense wouldn't be able to pick up which properties are available, and you'd have to introduce code that looks up properties and gets their values in runtime. You can do this with something called reflection. Doing so will impact performance, as well as making the code harder to read and maintain. To get all the properties on the anonymous type, you can first retrieve the type information. This will contain all the necessary information about the class. You can then get a list of all the properties. And with the property information, you can then get the value from the instance that contains this property. Hopefully, this is convincing enough that working with anonymous types outside of the current method is a bad idea. Don't use them as return types or pass them as parameters. That is not what they are intended for.

Summary

This module introduced you to the anonymous type. This is a great way of representing a subset of properties for a more complex type. It's very often used together with LINQ. An anonymous type is constructed with the new keyword followed by an object initializer. When you use the new keyword and the object initializer without the type, the compiler will know that this is an anonymous type. When you create an anonymous type, the compiler generates a class with the appropriate properties and methods for you. There is no way of explicitly using the generated class. And therefore, it's not appropriate to use the anonymous types when you need to return data or pass it as parameters. Its purpose is to make it easy to represent a subset of data used only in the current method. You can pass it around, but doing so will require reflection to find the properties and their values in runtime. This will impact performance. Using LINQ together with this language feature lets you, for example, select a new representation of a complex type that allows you to create a new list of elements with copied values. Remember that when it comes to copies of reference types such as arrays or lists or an instance of a class, it will only copy the reference, not the value inside the reference type. We saw an example of using LINQ and the anonymous type with the data grid in WPF. This resulted in an easy way to automatically create a representation of the data without having to introduce a custom class. Because anonymous types are read-only, you have to use a non-destructive mutation, which is achieved by using the with expression. The with keyword, followed by a statement, lets you copy the

instance of the anonymous type with one or more properties changed. As we've discussed through this module, and you've seen this firsthand, the anonymous types can be really powerful and reduce the amount of unnecessary classes. Whenever we can get the compiler to generate code to make our lives simpler, I see that as a win. There is another way to represent a group of values that will also be usable as a return type, as well as a parameter to a method. In the next module, we are going to look at a different way of representing a subset of properties, much like an anonymous type. But with this approach, it can be returned or accepted as a parameter. We'll be looking at a language feature called tuples or tuples, depending on how you prefer to pronounce them. As you'll soon experience, they're a whole lot more powerful than the anonymous type.

Tuples & Deconstruction

Introducing Tuples

Welcome back to the course C# Advanced Language Features. I'm Filip Ekberg, and now it's time to look at a different way of grouping a set of elements. As opposed to using anonymous types, this approach will let you use the definition of the group anywhere in the application. This language feature is known as tuples or tuples. How you pronounce this may vary. But nevertheless, we're talking about the same language feature. The concept of a tuple has been around in .NET for a long time. But in C#7, this became a first-class citizen. The definition of a tuple is that it's a sequence of elements. In C#, this will be a strongly typed sequence of elements. Creating a tuple is easy. You start off with two parentheses. Inside the parenthesis, you define the elements that you want to group into this tuple. If you specify that you want to create a tuple that contains the order number and a reference to the line items, you simply add those elements to the tuple. When the compiler sees this, it will generate some code for you. And internally, this will be using something known as a value tuple. The value tuple is a struct, but that doesn't stop it from containing reference types inside the tuple. You can use the var keyword to infer that this is a tuple. And when storing this in a variable, you can, of course, access the elements within this tuple. As you can see here, just like with the anonymous type, the names for the elements are inferred by the compiler. Remember that the anonymous type had to explicitly define the

names when performing a method call? That's not the case with a tuple. If you summarize the price and add the result to the tuple, the compiler will generate a name for you. In this case, since this is the third element, the name for this field will be `Item3`. This probably isn't the most appropriate name. We could explicitly define the name in the tuple. To do this, you use the same syntax as with named parameters. Specify the name followed by a colon and then the value. Now that you've seen how to construct the tuple and give explicit names for the elements in the tuple, converting an anonymous type into a tuple should be rather easy. There are some slight differences that are important to know. The anonymous type is a reference type, and the tuple is a value type. When you create an anonymous type, it will generate a new class that corresponds with the elements used in the type. If you compare that to the tuple, this will generate code that uses the built-in type value tuple. The anonymous type has all its elements stored as properties with names that correspond to either what's been inferred or the explicitly defined names. The tuple, on the other hand, uses fields on the generic value struct. The names inferred by the compiler or explicitly defined will only be tracked by the compiler. When your application is running, you will have no way of finding out what a tuple element was named, at least not without some custom code. Where this can make a huge difference is if you use serialization. Use the built-in support from `System.Text.Json` to get a string representation for this instance. You can see the order number, an array of items, and the total price. Sending this text representation to someone else, let's say a web API, that recipient of this text would easily understand what each of the different pieces are. If we do the same with a tuple, the result will be slightly different. As mentioned, the tuples store their values in fields. And by default, JSON serialization only serializes the properties. Therefore, we can immediately see a difference in how you'd serialize this. You'd have to explicitly define that you also want to include the fields when serializing the instance. Looking at the result, it's no longer obvious what each piece in the text represents. Surely, we know that `Item1` is the order number, `Item2` is the array of items, and `Item3` is the total price. Because the compiler translates the code into using the generic value tuple, the names are discarded after compilation. This makes sense when you look at the code that was generated. There is one more important difference between tuples and anonymous types. On anonymous types, the properties are read-only. For tuples, the fields are not. You can change the values after the tuple has been created. So far, a tuple doesn't look very different from the anonymous type and how that is used. We've only scratched the surface. You have now seen how to create a representation of a group of elements. We assigned this group to a local

variable, and the compiler automatically inferred some of the names for those fields. When the compiler failed to infer a good name, we could explicitly define that. While we used the `var` keyword to infer the type, we don't have to. In fact, you can define a tuple signature and the fields instead of using the `var` keyword. Do this to explicitly define what this group of elements will contain. Next, we are going to explore how this tuple assignment works in different scenarios, and we'll dig into deconstruction of a tuple.

Tuple Assignment and Deconstruction

You've already seen the most simple type of tuple assignment. On the right-hand side of that assignment, you have the tuple defined as a group of elements. Assign the tuple to a local variable and the type of the tuple will be inferred. Optionally, you can specify the type of each field in the tuple, which uses the same syntax as when we create the tuple. With this line of code, the tuple on the right-hand side will be mapped into a value tuple with the fields using the types defined. Changing this from `var` will behave slightly different. If you explore the fields available on the local variable, you can see that the compiler no longer inferred the names. To introduce names for each of the fields, you can explicitly define that. This means that the tuple of the right-hand side is now assigned to the tuple as defined on the left. There's a reason that both of these alternatives are supported. We'll look at a good use case of this when we introduce a tuple as a return type. For now, let's continue to talk about the different types of tuple assignment and how the tuple can be deconstructed. What's really great about tuples is that it supports different ways of assignments. If we completely remove the name of the tuple, this will still compile. But what exactly does this mean? Instead of the tuple being assigned into a local variable of type tuple with the fields that we've specified, it's now assigned to these three different variables. The order of the right-hand side and the left-hand side need to match. Otherwise, it wouldn't be able to assign the value from the tuple into the local variable. We can take this one step further and remove all the explicit types and have the compiler infer each of the types for us. Use the `var` keyword to do this. Since the compiler knows the type used in the tuple, it will be able to figure out what the types of the local variables should be. You can even take it one step further and only have one `var` keyword. This will create three local variables. The types will be inferred by the compiler, and the number of variables defined here need to match the tuple exactly. You can try to have a different number of variables on the left-hand side of this tuple assignment. Notice how the compiler tells you

that you cannot deconstruct a tuple with two elements into three variables. What we are doing here is not only assigning the right-hand side to the left, but we are doing something called deconstruction. Tuples have built-in support for this, which is what makes it possible to assign it in the different ways that we've looked at. Since the tuple can be deconstructed into multiple different variables, could we deconstruct the tuple into already existing variables? If you pause the video and try this yourself, can you figure out how to do this? If you are ready, let's take a look at how the tuple can be deconstructed into already existing local variables. Let's say that the application already had a variable for the order number and the sum, but it doesn't have the `LineItems` local variable. First, remove the `var` keyword from the start. This will tell you which of the variables do not exist in the current context. We can point the first variable to the order number and the sum to the local variable. I don't want to manually add a local variable outside of this definition. Do you know what we can do? You can add the `var` keyword. This will introduce a local variable with the name. And when the tuple is deconstructed, it will add the first value into the order number, which is defined above. It will map the line items into a new local variable. And finally, it will set the sum that was defined above to the value from the tuple. This makes working with tuples extremely pleasant. It's really much more flexible than what you'd get with an anonymous type. The code generated by the compiler will be similar no matter if you deconstruct into an already existing variable or if you deconstruct into new variables. Remember, it's really important that the left-hand side and the right-hand side have a matching number of elements. Otherwise, you'll get a compiler error. In some cases, when you retrieve a tuple, you may not care about one of the values, and you don't want to deconstruct that value into a local variable. This is where you can use what is known as a discard. You saw this with the lambda, and the same idea applies to tuples. You use the discard by adding an underscore, and this simply means that you don't care about this value. We could therefore specify that we want a tuple deconstructed into two local variables, order and sum, while completely ignoring the middle one, which is the `LineItems`. You can discard as many as you'd like. You could even discard all of the values or simply discard the entire return value. This would work for any assignment and is useful in cases where you explicitly want to say that you don't care about using that return value. Using discards can reduce memory allocations. As you've seen by now, tuples as a first-class citizen in C# is really a powerful language feature. The options you have in terms of deconstructing a tuple makes it useful in many situations. You've probably gathered by now that if you have a way of defining what type exists in the tuple, you're not limited to use this only in the current

method. Next, we'll be looking at returning tuples and accepting them as an input parameter to our method.

Using Tuples as Return Types or Parameters

We previously built a method in the `OrderProcessor` that returned an anonymous type. The problem with this approach is that whoever calls this method has a really hard time to get the values out of the anonymous type. Converting an anonymous type into a tuple should be rather straightforward. The anonymous type is created using the `new` keyword together with an object initializer, while the tuple is constructed without a keyword and a signature that looks like a method call, it's simply a way to define the grouping of elements. Let's attempt to refactor this method. First, we'll get rid of the `new` keyword. Then, we'll replace the curly braces with parentheses, and we're left with a compiler error because we also need to replace the equals with a colon. That seems to have gotten rid of all the compiler errors, and the code actually compiles. If we now hover the local variable summaries, which is the result of that `Select`, we can see that this is an `IEnumerable` of a tuple that matches what we've constructed. The compiler inferred all the types for us. The `with` expression works with a tuple because the tuple is a struct. You can use it together with structs, record types, and anonymous types. Just like with the anonymous type, this gives us a new instance with the modified values. Even though the tuple isn't read-only, it's good practice to work with an instance as if it's immutable. You would especially find this useful if you work in a multithreaded application. The method signature defines that it returns an object. So from the consumer point of view, using this method hasn't changed. How do we make it easier for anyone that uses the method to consume the result of the tuple? Can you guess? We do the same as when we explicitly declared the types that the tuple was going to be deconstructed into. Replace the return type with a definition of the tuple that we want to return. When this is compiled, the compiler will replace this with the generic value tuple. You could define that you return the exact value tuple as well, but that requires more characters and it's a little bit less readable. I prefer that the compiler takes care of this for us. A problem that you will experience now is that when you call this method, while it will return a tuple, the names of the fields on that tuple will not be inferred. This means you'd get `Item1`, `2`, `3`, and `4`. Just as we did earlier, we can tell the compiler which names we'd like to use. Let's say that the first value in this tuple is a GUID with the name `orderNumber`. While this won't have an effect when you serialize the tuple, it will show up in IntelliSense if you include this library in

different projects. This is because if we look at the generated code, the compiler applies an attribute called `TupleElementNames`. This will contain a list of strings for the field names that you have explicitly defined. Where you see the nulls, those are names that you have not explicitly defined. As you can see, you can define one or more of the names. It's a good idea to specify all of them so that you don't confuse yourself or your fellow developers that work with this code. This method is now completed. We've changed the method signature to allow us to return a tuple. The signature makes it look like we have multiple return values. But in reality, we are returning a tuple, which is a group of elements. Behind the scenes, it uses a value tuple, which is a struct, that will hold the values and references to the objects that we return. Let's head over to the program and see how this works on the consumer side of things. Accessing the fields show that they're displayed using their names as we specified in that method signature. If we make two subsequent calls to `Process` and store the result in two completely different variables, given that both of these tuples have the same values, what do you think is going to happen when we compare these two variables? If you recall, comparing the anonymous types using the equality operator resulted in a reference comparison. However, in this case, the value tuple isn't a reference type. Let's compare these two values and see what happens. As you can see, this comparison returned `true`. This is because comparing two tuples will perform value-based equality. We could try and change one of the values. And to make it simple, we can use the `with` expression. Then, we can rerun the application. It's now `false`. This is the expected behavior when using tuples. You can use all the different types of deconstruction that we've looked at earlier. You can take the result from the call to `Process` and assign that to a local variable. This is now used as a tuple. Or you can deconstruct it into multiple different variables or even deconstruct it into existing variables. Optionally, you can use the `discard` on the values that you do not care about. The same way that you can use a tuple as a return type, you can use it as a parameter on a method. This works with `action` and `func` as well. You can use an `action`, for example, that expects a tuple to be passed into the method. Invoking this delegate lets you pass a tuple as the argument. This works for all methods. You can use the tuple just like you did with a return type. Because the compiler uses the generic tuple behind the scenes, you could even create an extension method that works for all tuples with a specific number of elements and types. We can head over to the extension method library and add an overload to the `order` extensions. Maybe we want to introduce an overload that accepts a tuple because the tuple represents an order, and we can then produce a report based on that. Remember,

extension methods are public static methods. We'll return a string, and the type that we are extending is the tuple. We'll define the group of elements just like we did with the return type. We can give this parameter a name, and now we can access each of the fields of this tuple inside the method, return a report based on the tuple, and after we are done, we can head back to the program. What's interesting now is that this extension method will immediately work on what's returned from the method process. This is because the tuple that is returned matches the tuple that we defined the extension method for. This extension method will only work for tuples with the exact same definition. It needs to contain four elements that have a matching sequence of types. As an example of this, you can construct a tuple with an empty GUID, an integer that is 0, a decimal that is 0, and finally, an empty enumerable of Item. That will simply give you an IEnumerable of Item. As you see, the extension method GenerateReport would be available. But if you change the order of the types, it will no longer work. This demonstrates how to use the tuple as parameters to methods. And as a bonus, you added an extension method. If we look at the decompiled code for this extension method, it may become even more clear why and how this works. It's simply an extension method for value tuple of GUID, int, decimal, and IEnumerable of Item. There's one more change that I'd like to make before we proceed. The method Process accepts an IEnumerable of Order, but only returns one summary. I'd like to change this to return an enumerable of tuples. This change is simple. Change the method signature to indicate that you want to return an IEnumerable of the tuple that we defined earlier. Instead of returning just one summary, we can return the entire list of tuples. You're now working with an IEnumerable and not a tuple that can be deconstructed. To iterate through each of the items in the enumerable, we can simply foreach, and you'll notice that the assignments and deconstruction of the tuple works for each element in the enumerable. You can then proceed to call GenerateReport as long as it's not deconstructed into different variables. Working with tuples is powerful, and they provide much more flexibility in comparison to anonymous types. But when it comes to data binding, as we saw with anonymous types, the tuples cannot be used in the same way. So for that use case, you'd still have to rely on the anonymous types or create an explicit class for that purpose. Now that you've learned about how to define tuples and the ways that they can be deconstructed, it's time to look at how to deconstruct other objects that are not tuples and why this can be useful.

Deconstruct Other Objects

Wouldn't it be great if there was a way to take an instance of, for example, the order and define how this could be deconstructed into local variables, just like we did with a tuple. The compiler doesn't do this for you, but there is a little bit of code that we can add to make it possible to deconstruct a class, interface, or even a struct. This is useful for a couple of reasons. You will discover one of the most useful cases in the next module when we introduce pattern matching. We'll take a sneak peek at that in just a moment. The goal is to be able to take an instance and deconstruct that into local variables. Let's say that you want to take the order and be able to deconstruct this into two variables, one representing the order total, the other if it's ready to be shipped. To achieve this, you can create an instance method that's void and called Deconstruct. If it's void, how do you define which variables are deconstructed from this instance? This is done using out parameters. You have to specify at least two parameters. Otherwise, you cannot deconstruct the object. Out parameters always need to be assigned a value. Otherwise, you'd get a compiler error. Let's go ahead and add this deconstruct method to the order and see how we can use this in the program. This is a public void method with the name deconstruct. Naming this correctly is important. Then, make sure that you set all the parameters to out parameters and assign them to appropriate values. You could have anything as an out parameter. It doesn't necessarily have to be a property or field from this instance. It could be something that we compute or the result of an additional method call. For example, the parameter ready could be checking that both the IsReadyForShipment property is true, as well as there being a shipping provider specified. Let's just check IsReadyForShipment for now. And you can make additional experiments on your own. This deconstruct method is now complete. Because we've followed this pattern, the compiler will now know how to deconstruct this and generate appropriate code for us. Head over to the program and take an instance of an order, and we can assign the instance to two local variables using the same approach that we did with a tuple. There's a slight difference here. This cannot be deconstructed into a tuple. It has to be deconstructed into separate variables. When the compiler sees the var keyword followed by the variables that we want this instance to be deconstructed into, it will automatically generate code that simply calls the deconstruct method using these two local variables as the out parameters. It's really just a handy way of getting an alternative representation of this class. Hence, it's deconstructed into a different representation. There is a way to combine this with using tuples. We can ask if the instance of the order is a match to a tuple with constant values. This will, in fact, check if the order is not null and if the total is above 0 and that

it's ready to be shipped. Does this mean it would be possible to compare this against an instance of a tuple? The answer is no. If we look at the generated code, it may become clear why that's the case. How would you compare this to the generic value tuple, which would be generated if you defined a tuple? The code we used in the if statement is definitely something that we are going to explore in great detail in the next module. You may be wondering now if it's possible to have more than one deconstruct method. That is definitely possible. You just have to make sure that they have a different number of parameters. If you introduce an overload for Deconstruct, it doesn't matter if the parameters have different types if they have the same number of parameters. The call would be ambiguous, and the compiler wouldn't be capable of determining which one to use. We can add an overload where the final out parameter is the items. If you then head over to the program, you can now decide which way you'd like this instance to be deconstructed. By changing the number of variables that you are deconstructing your instance into, it chooses the appropriate method. If you like, you can even ignore one of the values by using the discard like we've seen earlier, just as with the tuple deconstruction and the lambdas. Even if you use discard, it will call the method with those number of parameters. This means if you capture two and discard one, it will call the method with three parameters. If you want to add the capability of deconstructing a type that you don't own, how do you think you'd do that? You can do this with an extension method. Let's add a deconstruct method as an extension method in the OrderExtensions class. Since this is an extension method, it will be a static method, still using the name deconstruct with the first parameter being the class that we are extending. Then, we can specify the out parameters, and they need to be a different number of parameters than what the other deconstruct methods have defined prior. This deconstruct will give us the order number, total number of items, the items, and an average price. By now, you should know how to call this. If you add an extra variable or discard, this will invoke the extension method. Being able to deconstruct an object into a local variable, you can make the code easier to read, which ultimately makes it easier to maintain and understand. As a final example of deconstruction, I'd like to highlight one of the use cases in .NET. If we have a dictionary of string and Order and we'd like to iterate through this, the generic key value pair, in fact, implements a Deconstruct method. This will let you deconstruct this key value pair into two local variables. Why does this make a difference? If we iterate through the collection, we could say that the key is, in fact, the order number and the value is an order. The type of the value would have been quite obvious if we looked at the dictionary, but having this deconstructed

into two local variables with explaining names makes a huge difference. Now you've seen how to work with tuples, how to return them, accept them as parameters, and how to work with deconstruction in different situations.

Summary

This module introduced you to tuples and how they are a first-class citizen in C#. The definition of a tuple is that it's a sequence of elements. You saw how to define this group of elements using a new syntax. You can give the different elements in the tuple names, which makes it easier to work with. When the compiler generates code for this tuple, it uses a generic value tuple where each of the elements are stored in fields. It's worth to remember that these are not read-only only. You saw the different ways that the tuple can be deconstructed and assigned. The most simple assignment is to store the tuple in a local variable and then letting the compiler infer the type for you. You can also deconstruct the tuple into new local variables. This can be done in a few different ways. You may explicitly define the types that you deconstruct the tuple into, or you can reduce the amount of characters and make this a little bit more readable. If you want, when deconstructing the tuple, the values can be assigned to already existing local variables. There might be values that exist in the tuple which you don't care about. For those situations, you can use a discard to indicate that you don't want that stored in a local variable. You have to do this because the left and the right-hand side of that assignment need to have the same number of elements. You then learned that the tuples can be used as return types or even as parameters to a method. Since we've seen what the compiler uses behind the scenes, it makes sense why this works. Use tuples as return types in situations where introducing a new class that represents the group feels unnecessary. To return a tuple from a method, you use what is known as a tuple literal. What is important here is the order of the elements. It doesn't matter what names you give them. If you do have a tuple return type, you should consider giving those elements names as this will make it easier for whoever is calling this method to know exactly what each element in the tuple is. They can then use this to assign or deconstruct the tuple into variables. If you happen to swap the names when deconstructing the tuple, the value will end up in the variable at the specified position. But now, it gets the wrong name. After learning about deconstructing a tuple into multiple local variables, we discussed that it would be beneficial to be able to do this for other types as well. If you create a void method on your class, interface, or struct that is called Deconstruct with a

number of out parameters, this will make it possible for an instance to be deconstructed into multiple different variables according to the out parameters. You need at least two out parameters, and you can overload this method with a different number of parameters. It's important to know that you are required to have a different number of parameters for those overloads. It does not matter if only the types of the parameters are different. As a bonus, you saw that you can introduce an extension method that adds a Deconstruct method to a class that you don't own. While we do own the order in our application, we'll just imagine that we don't. This means we could essentially deconstruct anything. We saw two really good use cases for deconstruction. One let you deconstruct a key value pair when iterating through a dictionary and map the key into a local variable with a better name and the value into another local variable. This makes the code more readable and easier to understand. The second case was with pattern matching where we used a tuple to define a pattern, which we could use to compare with an instance. The instance was deconstructed to match with this pattern. This is exactly what we are going to explore in great detail in the next module. If we have a way of pulling the object apart and looking at its different pieces and traits in different manners, wouldn't that be a very good way of determining what an object is and what it isn't. We can do this in many different ways, one of them being what you got a sneak peek off in this module. Next, we are going to learn all about pattern matching and how to construct simple, as well as complex patterns. This is definitely a language feature you will be using all the time.

Pattern Matching

Introducing Pattern Matching

Welcome back to the course C# Advanced Language Features. I'm Filip Ekberg, and now it's time to learn all about pattern matching. Pattern matching isn't a new concept. In fact, it's been around for ages and have been a part of programming languages such as F#, Kotlin, and others for a very long time. In C#, however, it's a fairly new language feature and was only first introduced in C# 7. At that point, the language feature was rather limited. As with many other language features such as tuples and lambdas, newer versions of C# may introduce additional improvements. This has been true for

pattern matching as well. Since its first appearance in C#, there has been a lot of improvements. Future versions of C# will most definitely continue to introduce new patterns that will make your development easier. So, what is pattern matching, and why is this such an important language feature? I'd like you to consider the order in the warehouse management system. As we pull this object apart and look at its different values, properties, how it's deconstructed, we might want to construct an expression that lets us determine what the order is. Is the instance of a specific subclass? Is it marked as ready for shipment? Is it a canceled order with an amount less than a certain value? All of these questions can be expressed using pattern matching. Prior to the introduction of pattern matching, you would have ended up writing a code snippet that checks a given type, looks at different property values, and then based on that, it would perform some operation. In the simplest explanation, a pattern is just another way of allowing you to express what you assume the object is and what that may contain. Normal if and else if statements can quickly become hard to read and maintain if you have to write complex checks. This is where pattern matching will be a tremendous help. Of course, as with any language feature, if you abuse it, the code may just as well become hard to understand. Take this simple piece of code as an example. We'd like to check if the type of the ShippingProvider used with this order is a SwedishPostalServiceShippingProvider. This class inherits from the ShippingProvider. So after we've checked that this is the provider used for the order, we'd like to consume this as a SwedishPostalServiceShippingProvider. Using the is operator will determine the type, as well as if it's not null. This will use what is known as the type pattern. An alternative way would have been to get the type from the instance by calling GetType and compare this with what you'd get from typeof. You'd have to manually cast the ShippingProvider to the correct type and then store this as a local variable so that it can be reused within the statement. With pattern matching, this becomes easier. The if statement isn't extremely complex at this point. But what if we'd like to check a property that only exists on the provider? Then, all of a sudden, you'd end up with a nested if statement. Using the operator is one way of using pattern matching. The right-hand side of the is operator is where you'd define the expression for the pattern. We can expand on the pattern to express much more complex expressions. For example, we can specify that we want this captured in a local variable of the correct type. To do this, we add what is known as the declaration pattern. This will mean that inside this statement, you can use the provider variable as a SwedishPostalServiceShippingProvider without having to perform any explicit cast yourself. We've

reduced the amount of code by a fair bit, and this becomes even more powerful when a more complex pattern is applied. Throughout this module, you will explore all the available patterns and how they're used in different ways. You will learn about the following patterns, type, declaration, constant, relational, logical, property, positional, var, discard, and parenthesized. All of these patterns can be used in three different ways. The first option is that you can use them together with the `is` operator as we've just explored. Second, you can use it with a switch where each case in the switch statement is used to a pattern. This can be used together with a case card known as the `when` keyword. The `if` statement that uses the `is` operator can be converted into a switch statement. If you'd then like to check a property, you can do so using the `when` keyword followed by what you'd like to check. This introduces a case guard. The order will be the actual type that you matched on and also available in the case block. The third and final way is to use this with what is known as a switch expression. This is what many developers will prefer as it's more compact and usable in many different situations. It also makes it easy to understand exactly what you are looking for and what the intent with each match is. Being able to use patterns in all these different situations means that you can write code in the way that you prefer and even easily convert old code into using patterns to make it more understandable. As you explore each different pattern throughout this module, you will notice that you may combine and use more than one pattern. This is known as recursive or nested patterns. An example of a recursive pattern could be that you first check the type. This will also ensure that it's not null. Then, after the type has been checked, we'd like to check a property value. Let's say that if the order has been set to use the `SwedishPostalShippingProvider` and the freight cost is more than 100, we'd like to add a discount to the order. What we've expressed here is, in fact, introducing a recursive pattern. First, it checks the type. Second, it accesses a property to verify a value. Third, it uses something known as a relational pattern that lets you check, for example, if the value is greater than your constant. I know, there's quite a bit going on here, and we'll look at each piece individually throughout the rest of this module. As you've now seen, pattern matching is powerful and allows for simple and complex patterns, which let you express the intent of your code in a more intuitive manner. Let's now continue to explore pattern matching in C#. And in the next part, we are going to introduce a switch expression.

Switch Expression

To make it easier to work with patterns and pattern matching. A language feature called switch expression was introduced. This combines the idea of a switch statement and the expression bodied members. The purpose of a switch expression is to use patterns to evaluate which value to return. The syntax looks slightly different from using a switch statement. As opposed to a switch statement, the switch expression returns a value. The switch expression has a few important components. First, you start off with where you'd want to put the result. This could be assigning it to a local variable. On the right-hand side of this assignment, you first indicate which value, tuple, or object you want to produce a switch expression for. With a normal switch statement, this is what you'd use in the parenthesis after the switch keyword. So really, they are just flipped around. This way of writing the code makes it more fluent to read as you clearly state that the result will be based on the object. And then, we have the different patterns. Each case consists of a pattern and an expression. The expression has to return a value. All expressions need to return types that are compatible with each other or that define an implicit conversion. For example, one case can't return an integer while the other returns a string. That really doesn't make any sense. The pattern can be simple or complex, and you can combine multiple patterns, which is known as recursive or nested patterns. It's important that your switch expression has patterns defined for all different cases. I'm definitely not suggesting that you produce complex patterns for the sake of adding more patterns, but you should never forget to catch the default case. This is when there's no other match. You can easily catch everything by using the discard pattern, and this will match on null as well. If you forget to add a default case using the discard and the switch expression does not define a pattern that matches the object, you will end up getting a switch expression exception saying that non exhaustive switch expression failed to match its input. You may want to produce an exception on your own if that's the case, but at least you intentionally handled that case. Just like a normal switch statement, the order of the cases matter. You want the most granular patterns at the top. If the compiler notices that you have a pattern higher up in the switch expression that is less granular, it will give you a compiler error as you'd have unreachable code. Let's say that we'd like to calculate the freight cost for an order with the help of a new method that uses a switch expression. There might be multiple business rules to change the outcome of how much it costs to ship an order. We can do this calculation in the OrderProcessor. Let's introduce a new method that returns a decimal and call this CalculateFreightCost. This will accept an order as the only parameter to the method. Introduce a new local variable called freightCost. And based on the order

ShippingProvider, we are going to determine if this matches a few of the patterns defined by the business. Inside the switch expression, we can now look at the ShippingProvider. If the selected ShippingProvider is the SwedishPostalServiceShippingProvider, this means we will use the type pattern to look at the actual type of the instance. Then, we are going to consider the shipping free. However, with the Swedish Postal service, if you've selected next-day shipping, we'd like to make it a little bit more expensive. We can do this by combining two patterns, the type pattern and the property pattern. First, we specify the type. Then to access a property, we use curly braces and then the property that we want to match a value on. If this is added after the other pattern, you'll notice that the compiler produces an error. It will tell us that this is unreachable because it's already been handled by a previous expression or if the compiler determines that this is an impossible match. The type pattern defined above is a less granular pattern. It will match on all the shipping providers of that type. It's easy to fix this. We can simply move the switch expression that we just added above the other one. Finally, we'll add a default case, and that is done using the discard pattern. For all other shipping providers that do not match our criteria, we'll set the cost to 50. We can then return the freight cost from the method. We've now successfully introduced our first switch expression. This makes the code easier to read, and it's easy to change and add more patterns. You don't have to store the result that is produced by the switch expression in a local variable. You can make this method a little bit less chatty by directly returning the value produced by the switch expression. We can even take it one step further. A huge benefit of the switch expression as opposed to a normal switch statement is that we can use this together with an expression bodied member. Making this change is easy, and with very minimum effort, we've changed this and it's now become an expression bodied member. You've seen a few of the patterns that you can use already. And together with the switch expression, this becomes a really handy language feature that will definitely change the way that we write our code. We'll now continue through this module and explore the different patterns that you can use

Type, Declaration, and Constant Patterns

The most simple pattern is the type pattern. This will perform a runtime check to determine which type it is. This is useful in many cases. One example is to check if you are working with a specific subclass. Depending on what you match on, the type patterns may not even have to be related to each other. Consider this code snippet where you have an object and you may want to determine which type this

is. You can then add a few patterns. For example, you can ask if it's an integer, string, decimal, or really any type that you'd like. Most of the times, you'd be working with a shared base type or have a constraint on a generic type. When it comes to generics, the compiler does not take the generic constraint into account. That is why you'd be able to add a type pattern for integer, string, or any other types that aren't really applicable to the constrained type. If you use a concrete type, a base class, for example, the compiler would produce an error if the pattern is not applicable in this instance. The type pattern is extremely useful. Consider if we'd introduce a subclass of the order that represents a cancelled or a shipped order. The type pattern will also ensure that you have an instance of what you want to match. If it's null, it wouldn't be a match, which makes sense because it looks at the runtime type of the instance. You can capture the value as a local variable with the correct type. That is the declaration pattern. We can modify the calculation of the freight cost to give this a try. The first pattern expressed in this switch expression first checks the type using a type pattern. Then, it applies a nested pattern to check a property value. If we'd like to capture this as a local variable that we can use in the expression, we'd simply write the variable name. Let's call it provider. We can then use this inside the expression as the concrete type. If we hover the provider, we can see that this is the `SwedishPostalServiceShippingProvider`. To make the code a bit more interesting, we'll use the property `FreightCost` on the provider and add an additional cost if you've asked for next-day shipping. For all the other instances of this `ShippingProvider`, hence all the instances that do not express next-day delivery, they'll get a bit of a discount. For the default case, this is all the other shipping providers, how do we access them inside this expression? We can't use the discard pattern because discard means that we don't care about the value. This is where we can introduce the var pattern. The var pattern much like the discard will match on everything, but it is slightly different. It will allow you to capture the variable that was used in this switch expression. And then, you can use it in the default expression. Why is this useful, and why not just use `Order.ShippingProvider`? Well, consider the case where that call could be expensive or perform some computation. Doing that again could have some performance impact. The var pattern will capture everything that is not caught by the patterns above. This is just like the default case. You've now seen how to use the type pattern, declaration pattern, and var pattern.

Positional Pattern

In the previous module, we introduced the Deconstruct method that allows for the order to be deconstructed into local variables. A huge benefit of doing this work is because it's valuable when it comes to pattern matching, we can use what is known as the positional pattern. The pattern that you produce is built the same way as you'd define a tuple. The difference is that this group contains patterns. Let's say that you have the order. The class introduced the Deconstruct method for two variables, the total and if it's ready to be shipped. This means you could take the order and match it against a positional pattern. Is the total the constant value of 100, and is it ready? Then, we'll enter the if statement. Using the constant pattern for the total wouldn't be very flexible. I'd like to say that whenever the order is ready, despite what the total is, this should be a match. To do this, can you guess what to do? If you guessed using a discard, you are absolutely correct. To make it super clear what the second parameter is that needs to be true, you can use a named parameter. This makes it even easier to understand. This is just a very simple example of the positional pattern. It verifies the position of what is produced when deconstructing the object. If you're just using this to check one property value that you could access directly through the instance, this would have been a rather complex approach. I'd like to produce a string that represents the current status of the order, and we can do this inside the report generation that we built earlier. I'll introduce a local variable that will hold the order status. This will be a string when we are done. Based on the order and how it's deconstructed, we can now add multiple different positional patterns. First, if the total is more than 100, this is going to be considered high priority. Second for any other order less than or equal to 100, if it's ready, we can return a string that tells us that the order is ready. Third, if it's not ready, we'll just say that. The default would only be reached if the order was null. In that case, we'd have a much bigger problem, and the code further down would fail anyways. Nevertheless, we'll add a case to handle this. We could throw an appropriate exception. The positional pattern requires a Deconstruct method to be available or that what you match on is a tuple. While the second parameter is a Boolean that comes out of the deconstruct method on the order, it may be the result of a complex calculation that wouldn't necessarily have been easy to do with normal property access. In terms of using the declaration pattern to capture variables, we have a few options. The positional pattern would allow you to use the var pattern to capture the value. This is just like the discard and specifies that any value is fine. But you'd like to use this value inside the expression. You can also declare a variable name after the positional pattern. What do you think this will contain? Will it be a tuple produced from what was

deconstructed, or will it be the entire order? It will, in fact, be the entire order. Capturing this as a local variable lets you use it in the expression. As you see, the patterns are extremely flexible, and expressing yourself using patterns can make the code easier to understand. Another example of the positional pattern is to use it together with a tuple. Let's also add a `ShippingProvider` status to the report. Instead of using the order, we'll construct a tuple with a number of elements that we are interested in. This tuple will contain the `ShippingProvider` and the amount of items that are being shipped, as well as a `Boolean` that represents if it's ready to be shipped. We can now produce patterns using the positional pattern to check each of the different elements. For example, no matter the `ShippingProvider`, if the amount of items that are being shipped or more than 10, we need to divide this into multiple shipments, no matter if it's marked as ready or not. If this is the most important case, it needs to go on the top. For the `SwedishPostalServiceShippingProvider`, if it's just one item, we are going to require this to be picked up manually. For all other shipping providers and all other quantities of items that are less than or equal to 10 and being marked as ready for shipment, we'll just say that this can be shipped from the warehouse. We can now add a final positional pattern for the case when the order is not ready. This is, in fact, the same as adding a `discard`. Personally, I prefer to use the `discard` in this case. The positional pattern uses nested patterns to match the position of the element in either what was deconstructed or in the tuple. Remember, the position matters, hence the name of this pattern.

Property Pattern

You may not always want to rely on the way the object is deconstructed to write a pattern that matches a given property. In the `OrderProcessor` where the freight cost is calculated, you saw that the property pattern was being used. The property value is matched against something known as a constant pattern. This used a `Boolean` to match the value of the property. Other constant patterns could be, for example, numbers, characters, string literals, enums, or a constant field. The property pattern does not have to be used together with a type pattern. In fact, the property pattern will ensure that you have an instance and then let you write nested patterns for the properties of the type. Notice what happens if we remove the `SwedishPostalServiceShippingProvider` type before accessing the property `DeliverNextDay`, removing the type produces a compiler error. This is because `DeliverNextDay` is a property that only lives on the type that we defined before using the type pattern. The only properties

shared amongst all the different shipping providers can be seen if we explore the available properties. So what's really interesting is that we can add the type pattern back, then use this nested pattern to apply a property pattern. We can access properties of the actual type. This is why `DeliverNextDay` is available. If you first write `if order` is followed by open and close curly braces, this could ensure that the order is set to an instance, a rather interesting way to perform a null check. You can even check properties of properties. For example, if the order is an instance and the property freight cost on the `ShippingProvider` is more than 100, we'd like to evaluate this expression. This combines the type pattern, property pattern, constant pattern, and the declaration pattern. You're not limited to just checking one property. You can write patterns for multiple properties by simply adding one after each other separated by a comma. This is extremely useful if you don't want to rely on deconstruction. And in a lot of cases, using the property pattern makes it much more readable. It really depends on the situation. As with any language feature, you should use it with caution. And I really can't stress that enough. Overusing or misusing pattern matching can lead to completely unreadable code. In most cases, you will find that it's absolutely pleasant to work with and does, in fact, make the code easier to work with.

Logical and Relational Patterns

C# also introduces a set of logical patterns. This works very much like when you write an `if` statement that uses the operator `and`, as well as the `or` operator. There is also a negation pattern. These different logical operations are called negated not, conjunctive and, disjunctive or. A good example of using the not pattern is together with a constant pattern where you check if an object is not null. You can easily state that if the order is not null, you'd enter the `if` statement. The not pattern can be applied in front of any pattern to negate it. While you could state that a Boolean value should not be false, it would most definitely be a better idea to state that you are looking for a Boolean that is true. If you have a property pattern that first checks if you have an instance, then goes ahead and check if the `ShippingProvider`'s freight cost is more than 100, if you negate this using the not pattern, this has a bigger impact than you might imagine. It will, in fact, enter this expression if the order is null. Think of it like this. It tries to evaluate if the entire pattern is true and then negates that. Be careful as the code could quickly become hard to understand or have unexpected behaviors introduced. This is just like negation in any `if` statement. We've used another pattern a few times already without talking about what that is. I'm

referring to the relational pattern. This is an extremely nice addition and a truly useful pattern. The relational pattern requires you to use a constant value to compare against the related value. If we head into the order extensions and look at `GenerateReport`, we've used the relational pattern at a few places already. We've used it in the pattern that deconstructs the order to determine its status. Here, we looked for high-priority orders by comparing the first value that comes out of the deconstruction with a constant value of 100. The relational pattern works with any relational operator, that is greater than, less than, greater than or equal, or less than or equal. The constant value has to be a number, character, or enum. You're not limited to use the relational pattern with the deconstruction or a tuple. It could, of course, be used with a property pattern as well. The relational pattern provides a good opportunity to introduce the patterns conjunctive `and`, as well as disjunctive `or`. Doing so allows you to define a range. Let's say that we'd like to introduce another level of priority for the order status. To do this, we can add another case in the switch expression. Start off with the relational pattern to define that this needs to be at least higher and 50. And then we'll apply a logical pattern using a conjunctive `and`. This means that it has to be more than 50 and less than or equal to 100. Finally, we'll say that the second value has to be true. Just because we've used the positional pattern previously in this switch expression doesn't necessarily mean that we have to do that for all the different cases. We could rewrite this to use the property pattern instead just to see the difference. We'll access the property. Then for the property total, we can define that this needs to be more than 50 and less than or equals to 100. We can create an order and generate a report on an order where the total is 75. It should now say that this is a `PriorityOrder`. On the other hand, if we change the order total to 101, this would instead be a high-priority order. You're not limited to just two values, and you could use this to exclude values in the range that you build a pattern for. While this doesn't make much sense for the total property, we could still give this a go and see how it works. Let's say that if the total is between 50 and 100 but not when it's 75, this is considered a `PriorityOrder`. This lets you exclude the constant value of 75 in relation to the property total. There's one more logical pattern. Since we've got a conjunctive `and`, there has to be a disjunctive `or`. Before we use this pattern, let's introduce a few subclasses of the order, a `PriorityOrder`, a `ShippedOrder`, and a `CancelledOrder`. Now let's go back to where we generate the report and use these subclasses in our patterns. At this point, the patterns don't care about the type as long as they can be deconstructed into something that matches the positional pattern or if it has a property that matches the pattern we used with a property pattern. If a class is

represented as a `PriorityOrder`, we may as well add that as a type pattern as well. If we simply add the type in front of the positional pattern, this would mean that it has to be of type `PriorityOrder`, as well as being deconstructed into what matches the positional pattern. This is effectively the same as adding an `and` between the type and the positional pattern. How about if we'd like to say that the order is a priority if it's using the type `PriorityOrder` or that it can be deconstructed into something that matches the positional pattern. This is where you can use the `or` pattern. It's important to know that the pattern on the right-hand side of the `or` is not necessarily going to be a `PriorityOrder`. Another important aspect is that when we use `or`, as well as `not`, you cannot declare a variable that the matched object will be stored in. Let's jump over to the program and give our new pattern a go. We'll construct an order and generate a report. If the total is above 100, we should still get this as a high-priority order. This works just like the `or` operator in an `if` statement. A problem with this switch expression is that it will match on a `PriorityOrder` or any type of order that simply has a value greater than 100 and is ready to be shipped. I'd like to express that this is either a `PriorityOrder` or a type of order that can be deconstructed into what we had earlier but with an additional requirement. It cannot be cancelled or a shipped order. To do this, we are going to group the second pattern into what is known as a parenthesized pattern. This way, we can evaluate this entire group of patterns together. We'll start by wrapping the positional pattern with parentheses, and then we'll append patterns that has to be matched. Therefore, we'll use the conjunctive `and`. This next part is very interesting. We'd like to define that the type is not a `ShippedOrder` or `Cancelled`. We can use the disjunctive `or` to specify the two types we'd like to match on and then negate that entire pattern. If we also add the name parameters to the positional pattern, this shows a complex pattern that is fairly easy to read and understand. We'd like to match on all priority orders or orders that have a total value above 100, are ready to be shipped, and are not already shipped or cancelled orders. Expressing this using a traditional approach wouldn't have been as easy on the eyes. Of course, this is starting to get a bit complex. And an easy approach in this case would have been to handle both the shipped and cancelled orders using a type pattern above this one. Even though this switch expression is getting a little bit complex and out of hand, it illustrates the power of pattern matching. You can combine a lot of different patterns and use it to express yourself in ways that have been a little bit more difficult before.

Summary

This module introduced you to the concept of pattern matching. This is a great addition to C#, and it changes the way that you express yourself. Simply explained, pattern matching is a way to construct an expression that lets you determine what an object is and what it isn't. It allows you to check, for example, how the object can be deconstructed, what properties has certain values, which subclass is used in runtime, and a whole lot more. Being able to express the intent with less ambiguity can be the difference between a great method and a buggy one. Pattern matching helps with this by introducing different ways to inspect the object and different ways to write patterns. Throughout this module, we explored the different patterns available in C#. I can almost guarantee that newer versions of C# will get even more patterns added for more common situations. We learned about the following patterns. The type pattern lets you write a pattern that checks which runtime type the object is. The declaration pattern lets you map the matched object into a local variable of that type. The constant pattern lets you write a pattern that compares a property or position or tuple value using a constant value. This could be a constant defined in your class, a number, a character, or a string literal. The relational pattern lets you use the comparison operators to check if the value matches your constant. The logical patterns let you introduce negated not, disjunctive or, as well as conjunctive and patterns. These three different logical patterns can be used together to produce complex yet readable patterns. You could check that a property falls within a certain range and that the type does not match two different subclasses. Another very good use case of the negated not pattern is to check if an instance is not null. The property pattern lets you check if the property has a certain value by allowing you to write a pattern for that property. This is a nested pattern that will first check if you have an instance and then evaluate the pattern for the property. You could use this in combination with the relational pattern or with other patterns as well. The positional pattern lets you write a pattern that matches on the values that are produced when deconstructing a type. It also lets you write a pattern that matches the value inside a tuple. The var pattern lets you capture the object into a local variable. It will be captured even if it's null. This is just the same as a default case, but where you'd want to use the value. The discard pattern lets you define a default case or simply state that you do not care about capturing the value into a local variable and that it can be of any value. Add this to the end of your switch expression and you'll ensure that you handle all cases. Finally, the parenthesized pattern allows you to group different patterns. You could define that a type is not one or the other. The patterns can be used in combination with case guards in cases where you'd want to check, for example, the type and then validate that a

method returned a certain value. You could then do this using a case guard. Pattern matching is truly powerful. There's a reason this has existed in many programming languages for a long time. It makes it easier to express yourself. And ultimately, it can make your code more understandable. However, as with any language feature, you should use it with caution and always consider if others understand the code just as well as you do. Going forward, do we have to add the Deconstruct method to all our objects ourselves? And what about value-based equality? This is something that we often have to add ourselves. If every developer has to do this, that seems a little bit unnecessary. Isn't there a way to automatically get all the functionality in a class that we normally must add ourselves? Next, we'll convert the Order class into a record type. This will introduce a whole lot of functionality behind the scenes while reducing the amount of code and headache.

Record Types

Introducing Record Types

Welcome back to the course C# Advanced Language Features. I'm Filip Ekberg, and now it's time to introduce records. The record type introduces a compact way of creating a reference type with a whole lot of useful functionality out of the box. You can use a record where you'd normally use a class. Creating a record requires much less code than creating the equivalent class. Let's take a look at how a record is created. It all starts with the record keyword. This keyword represents a record class. You could also specify a record struct. This determines if it's a reference type or a value type. The record keyword alone is the same as writing record class. Just like a class, you can give this type a name. Next is where we introduce a completely new concept that makes this very different from creating a class. The record is most often created with something known as a primary constructor. This uses a positional syntax to define the properties of this type. Each of the properties defined in this primary constructor will be a property on the type, as well as a required parameter on the constructor. You complete their record definition by ending with a semicolon. You can add properties, fields, methods, and everything that you'd normally use in a normal class by introducing a body, but let's explore that later. For now, the record is completed. What we've just introduced is known as a positional record.

This simple and concise syntax generates a class behind the scenes with a constructor and the properties that we defined. This is a reference type, and we can construct an instance of this record just like you'd do with a normal class. The primary constructor means that you are required to pass these parameters when creating an instance of this record. How is this really different from introducing a new class? What code is generated behind the scenes? And why would this be a better alternative than using a normal class? To find out, we can look at the record and the decompiled code. You'll first notice that this is, in fact, a class. It implements `IEquatable`, which indicates that this class generates some code that handles comparing one instance to another. The record gives you value-based equality out of the box. This is really one of the benefits of using a record. Next, you'll see all the backing fields for the properties as defined in our primary constructor. These are all read-only. This makes the record immutable as long as you don't introduce additional properties that aren't defined in the primary constructor. This means you can't change the values of these properties and their backing fields once you've instantiated an object. Knowing that an object is immutable is, again, really helpful in multi-threaded applications. You'll see the constructor and that it takes the values in the order defined by the primary constructor. Further down, we can see that the record overrides `ToString` and has a method called `PrintMembers`. This means we can get a string representation of the record and its values. This could be very handy in logging. Then, the record introduces the equality operators. This will use the `equals` method defined in the interface `IEquatable`. If we look at that implementation, we can see that it compares the type through the equality contract and then compares the backing fields from the two instances. Do you know what this means? This means we have value-based equality. When you compare two instances of the record, it will compare the reference. But if they're not similar, it will also look at the values contained in both instances. The record also introduces a copy constructor. Finally, at the bottom of this generated class, we can see a familiar method, and here we have a `deconstruct`. This deconstructs the record with the same out parameters in the same order as defined by the primary constructor. This means that if we have an instance of this record, it can easily be used with pattern matching and the positional pattern. We could, of course, use the property pattern with the record as each of the properties defined in the primary constructor are also accessible. Let's construct two instances with the same values and compare them. As long as the values are identical, the equality check will return true. What happens if we introduce inheritance? We can create a record that inherits from the previous record, and this requires that you pass the

parameters to the primary constructor defined by the record that you inherit from. You can use constant values, or you can add them to the inherited record and pass them along to the base record. These two records have the same properties and backing fields, and the properties live on the base record. What happens if we now compare an instance of the base record with the inherited record? This comparison will result in false. This is because the record uses an equality contract, which basically uses a reference to which type the record is. Since the types are not equal, it won't even go ahead and check the backing fields. This may come as a surprise and, therefore, it's important to keep in mind. Because records are immutable, we cannot change the values of those properties defined by the primary constructor. If the primary constructor happens to use a reference type, there's no guarantee that this reference type is immutable. As an example, consider if the record has a property defined that accepts a list. There's nothing that prevents anyone from adding more elements to that list. The only thing that is immutable in this case here is the reference to that list on the record. You cannot dereference the property on the record to point to a new list. If you'd really like to change a property, how do you do that? We've seen immutable types previously in this course. So by now, you might have guessed that we can introduce the `with` expression. Just as with the anonymous type, you declare which instance you want to copy and which properties that you want to change. The result is a new instance. Records are a very welcome addition to C#. It makes it easy to remove the boilerplate code that we generally have to introduce ourselves. This compact syntax of defining a reference type is not only easy to read and understand. It is also very powerful. While records can be used in most scenarios where you'd use a class, some libraries may have restrictions and rely solely on reference equality. As an example of this, you cannot use records in Entity Framework to define your entities. What you can do is use records throughout the rest of the application to define, for example, domain models. Records should be used when you want to represent data, not for classes that handle business logic. Records come with great functionality out of the box. But what if you want to override the behavior of one of the methods or introduce completely new methods and non immutable properties? You can definitely do that, and we'll explore this next.

Adding Additional Members to a Record

Sometimes the primary constructor isn't going to be enough. Just like when you define a normal class, you may want some flexibility. One way of looking at the primary constructor is that it's the least

amount of information needed to create an instance of the record. It doesn't mean that it's not allowing you to provide additional information or functionality. Just like a class, you can create a body for the record. This will allow you to add fields, properties, and methods just like you'd do in a normal class. This makes total sense because under the hood, the record is a generated class. Instead of ending the definition of the record with a semicolon, we are going to define a body for the record. We can add an expression bodied member that uses the two properties defined by the primary constructor. This can be used to concatenate multiple values or perform computation on those values. Let's say that we have a record that represents a customer where the first name and last name are two different properties. We might have an expression bodied member that returns the full name. The customer can also have an optionally defined address. Therefore, it wouldn't make sense to add this to the primary constructor. Instead, we can add a property that holds the address. This could, of course, be using a record that defines what an address needs to contain. If this auto property has a getter and a setter, that would mean this property is not immutable. The only immutable parts of the record are those that are a part of the primary constructor. How would we go by making this auto property immutable? That is, only being able to set this to a value when we create an instance. There is a keyword called `init` that does exactly this. The `init` keyword works with properties on normal classes as well. They're not limited to records. If you change `set` to `init`, this auto property is now immutable. It didn't add a new constructor or change the current one. How do you then set a value on this optional property? The beauty of using `init` is that you can initialize the value using an object initializer. Using the target type `new` expression, you do not have to explicitly declare the type that you are creating. The compiler will infer that this is an address, and you can pass the necessary values to the primary constructor defined by the address record. If you try to change this `init` only property after the object has been initialized, you will get a compiler error. This is now immutable, and the value can only be set in the initialization or when using the `with` expression to get a new instance. Because both the customer and the address are records, you'd get value-based equality out of the box. This means if you compare to customers, which will, in its turn, compare the address, the values will be used in the comparison. The address will also be a part of this value-based equality because when the compiler sees that an auto property is used, it will also compare its backing field in the equality check. You're not limited to adding properties or expression bodied members. You can, of course, introduce your own methods as well. Remember though that records should be used to represent data and when you need value-based

equality. If you find yourself in the need for other types of methods that don't just represent your records data in a different manner, in most cases, you should then have used a class instead. Most of the time it will suffice to use the primary constructor, but now you've seen how to introduce a non immutable, as well as an immutable property that uses the `init` keyword. While the `customer` and `address` are great additions in the warehouse management system, I believe we have classes in the application that are suitable to become records. Next, we'll be converting some of the classes that are better suited to be records.

Converting to a Record

Our warehouse management system has a few classes in the domain project that are really just data containers. These represent different domain models, and they're suitable to be refactored into records instead. Let's jump into the `Order` class and see how this can be converted into a record. If we change the order from a class to a record, notice what happens with the subclasses. We're getting a compiler error telling us that inheriting from a record requires this type to be defined as a record as well. This is so that the compiler can generate all the necessary code that is compatible with the base type. We can easily fix this by changing all these subclasses to be record classes instead. Inheritance is only applicable to record classes. Clearly, this change wasn't enough. We're still getting a few compiler errors. This is because the base type `Order` has code that we need to remove. Because we now define that this is a record, we've already got the equality operators overloaded. If you inspect the error on the operator, it will tell us that the `Order` already defines the member equality. We told Visual Studio to generate all this code for us previously in the course. Is it safe to remove these now? I know for a fact that we can safely remove this. If you do this in a real project, this is a great opportunity to introduce tests if you do not already have that. This will help you to ensure that you don't break your code. We'll remove the methods that we generated earlier, which is the equality operators, the two `Equals` methods, and `GetHashCode`. We can also remove the interface `IEquatable` from the record definition as this is already going to be added by the compiler when that sees that this is a record. If we compile the application, everything should really work the same as it did before. There should be no more compiler errors. Once this is compiled, we can look at the generated code to see that it did, in fact, generate most of the code that we removed. We can see the equality operators, the `Equals` method, and `GetHashCode`. We didn't have to do more than just mark this as being a record, and we

got value-based equality automatically. This is based on all the properties available on the record, just as we saw earlier when adding our own properties outside of a primary constructor. As you can see here, the methods and overloads that we have on the order will still work. Because we haven't defined a primary constructor for the record, there will be no generated Deconstruct method, although the Deconstruct methods we defined ourselves earlier will still work. This means all the patterns that we built earlier when introducing pattern matching won't have to be changed. If we want to go ahead and introduce a primary constructor for this record, this will require a fair bit of refactoring. Let's give this a try to get a feeling of how much work we'd have to put in. First, let's comment out any of the current constructors on the order. The only thing the default constructor did on the order is setting the order number to a valid GUID. We can do this with an auto property initializer. The order number is now immutable and can be set by an object initializer. If there's no value set in the object initializer, a new GUID will be generated. It would then be appropriate to require the rest of the properties to be passed into the primary constructor. The sole purpose of the order is to contain the data of an order. That is why a record is a suitable type. We'll ask for the Total, ShippingProvider, Items, and if it's ready to be shipped. If we want one of these to be optional, we can set it to a default value. We can specify that, for example, the default value for IsReadyForShipment is true. The contract of how an order is created has now changed. We've introduced a constructor that requires these values to be passed to it when instantiated. Have a look at the records that inherit from the order, and you can see that we are getting a few compiler errors. They all have to pass the correct number of parameters to the primary constructor on the order. Thus, we have to ask for those parameters to be passed into these records as well. This is just the same as when working with classes and inheritance. The subclass record can accept more or fewer parameters in the constructor as long as it calls the base constructor with the appropriate number of parameters. For the ShippingProvider and the CancelledOrder we are going to skip IsReadyForShipment and hard code the value false to be passed to the base class. We no longer have any compiler errors when it comes to the records and their implementations. But since we introduced a constructor on the type, we have to go through the program and fix those errors. Let's fix a few of these together. And then, I suggest that you pause the video and fix some errors on your own. In the program where we create an instance of an order, we have to use the constructor. This means we have to pass a total, an instance of a ShippingProvider. If we don't want a specific ShippingProvider, we can use the target type new expression. Then, we pass the items that are

related to this order. Now, I suggest that you pause the video and go ahead and fix the rest of the compiler errors on your own. If we had worked in a really large application, it might have been too much work to do all of this at once. An alternative approach would have been to set the parameters of the primary constructor to default values. If we do that, you can still set the values in the object initializer because the compiler generates init-only properties. Let's try that and change all of the properties defined in the primary constructor of the order record to have default values. If we now construct an Order instance, we can do so with the object initializer. Ideally, you'll want to ensure that all the data has been set, but this approach lets you refactor parts of the application without getting a million compiler errors. If you feel the need for multiple constructors, you may be better off using optional parameters. We've now refactored the Order class to an order record. This reduces the complexity and the amount of code in the application.

Serializing and Deserializing a Record

It's very common that you want to represent an object as JSON. I'm happy to tell you that you don't have to do anything in particular to convert a record to a JSON representation. Let's try this on an instance of the order. We can use the JsonSerializer from System.Text.Json to convert an instance of the order to a string that contains the JSON serialized object. Call the method Serialize and pass an instance. To make it easier on the eyes when this is printed to the console, we can supply an option to write the JSON as indented text. We can see a very nice representation of the entire order as JSON. We have everything that we need to convert this back to an instance of an order. Without any additional changes, we can deserialize this text into an instance. This is done by calling the generic Deserialize method on the JsonSerializer. We specify the type that we assume this is going to be deserialized into. And if we run the application with the debugger attached, you can see that we got an instance of an order with all the data intact. So far, this works just fine. But what about if we'd like to specify that when this is serialized or deserialized, the JSON property, that is the property name used in the JSON text, has a different name than what we use in our class? How do we define that? Normally, this is achieved by applying an attribute on the property. However, if we try to do that in the primary constructor on one of our properties, we are going to get a compiler error. This is because we're not, in fact, pointing to the property in this case. To explicitly define that this attribute is for the property that is generated for the total, we'd have to append a property colon before using the

attribute. We still apply this using the same approach as a normal attribute, but this lets you define that you intend to put this attribute on the property that is generated. If we compile this and look at the generated code, we can see that the attribute was applied to the property in the class. Another example of this is if we'd like to ignore one of the properties when serializing and deserializing. This is done with the `JsonIgnore` attribute. Use the same approach as when adding any attributes and then explicitly specify that this is for the property and that this will be ignored when this is serialized to JSON. Now that we rerun the application, the JSON property has a different name, and the property we ignored it removed from the JSON. Will this now work when it tries to deserialize this back to an instance of an order? Of course. We can see that this definitely worked. This has shown how to serialize and deserialize a record, but most importantly, how to apply attributes to the properties when defining your record.

Override Compiler Generated Methods

The compiler generates a set of methods for the record. It's good to know how to change the behavior or override these methods if you need to. Most of the time when you want to change one of the generated methods, it's how to represent the record as a string. When you call `ToString` on a record or a subclass to a record, this will in its turn use the method `PrintMembers`. If we create a string representation for the `Order` and for a `CancelledOrder` order, we can see that the string generated contains the name of the type and the properties. Some of them have values. But if they are reference types, they will only display the type that it is referencing. If we head into the `CancelledOrder` record and look at what we can override, we can see that we have the following options. We can override `EqualityContract`, `Equals`, `GetHashCode`, `ToString`, and `PrintMembers`. `ToString` uses `PrintMembers` internally, and `PrintMembers` will call its base class to get its members before applying the members for the derived record. You can experiment with overriding either `ToString` and `PrintMembers` on your own. This works just like overriding methods in general. Now, if we want to change them in the record order, which is the base class, how do we do that? Let's check if we can override these methods. You'll notice that on the base class, all that we have available to override is `GetHashCode` and `ToString`. Does this mean we cannot change how `PrintMembers` work on a record unless it inherits from another one? Of course not. We just have to follow the convention as specified by the documentation for records. To create a custom `PrintMembers`, we must introduce this as a protected

virtual method that returns a Boolean. And obviously, the name is `PrintMembers`. Then, this method accepts a `StringBuilder`. It accepts a `StringBuilder` because that is created by `ToString`, which is in charge of calling this method. You can then append the data to that `StringBuilder`, and this will then be used when you call `ToString`. Since this is a protected virtual method, it can be overridden by records that inherit from this record. With this overload added, call `ToString` and inspect the result that you get. If we run this, we can see that it entered our `PrintMembers` method. You've now changed the behavior of one of the methods that is normally generated by the record. Most of the time, you will be just fine using the prebaked functionality. But having the option to add additional details to the string representation can be crucial when it comes to logging.

Summary

This module introduced you to the very compact yet powerful way of defining reference types with the purpose of containing data. A record is created by either creating a record class or a record struct depending on where the data is stored. A big difference is that a record struct is not immutable by default. You can instead declare that you have a read-only record struct to make sure that this is read-only. If you do not explicitly declare either a class or struct, the record created will be a class. Hence, it's a reference type. Declaring a record is normally done with a primary constructor. This creates a positional record. The properties defined in this positional syntax creates backing fields that are read-only. This produces an immutable class. Out of the box, the record makes it simple to get a string representation of the content. It also introduces value-based equality. This means that you no longer have to write the cumbersome boilerplate code for each of your domain models. That is, the classes that represent your data. You can extend a record by defining a body for it. This allows you to introduce non immutable properties, methods, and other types of members. Keep in mind that the records are most suitable to represent data and shouldn't be used for the classes that contain your business logic. Those types should be normal classes. A good rule of thumb is to use a record when you need value-based equality. It wouldn't make much sense to use a record for the `OrderProcessor`. That class contains business logic that operates on the domain models. Therefore, the models can be records, while the processor is a class. If it's immutability that you are looking for, remember that you can still create classes with immutable properties by using the `init` keyword. You learned about record inheritance and overriding the generated methods. This provides great flexibility. We also discussed

that since the record comes with deconstruction out of the box, it can easily be used with pattern matching. You don't have to explicitly add a deconstruct yourself, but you could add alternative ones if you so like. Instead of introducing multiple constructors, you can specify that the primary constructor has optional parameters. We saw that this made it easier when we refactored the application to avoid a ton of compiler errors. This means we can gradually improve the application and refactor one part at a time. You learned that you can apply attributes to configure how the JSON is produced and deserialized. Adding an attribute on a variable in the primary constructor didn't work right away. We had to explicitly declare that we wanted that attribute added to the property that was generated for the variable. Records will play a big part in the applications that you build, and they will definitely make your life easier. With reference types being used in the application and the fact that you can set them to null, do you see a potential problem with that? It's just a matter of time before you run into what is known as a null reference exception. That is, when you try to access properties or call methods on a reference type that is null. With all the help we get from the language, surely there must be something that lets us find these potential issues before it becomes a problem. Next, we are going to explore nullable reference types and how it will help you avoid null reference exceptions.

Nullable Reference Types

The Problem with Nullability

Welcome back to the course C# Advanced Language features. I'm Filip Ekberg. And now, we are going to look at how to prevent null reference exceptions by using what is known as nullable reference types. If you've worked with C# for a while, you may have run into a situation where an object is not set to an instance before you try to access the properties or methods. This can cause the application to crash if it's not handled correctly. If we had the capability of traveling back in time, I'm certain that the language designers of C# would have considered making it much harder to use null. Unfortunately, time travel is not possible. So instead, we have a language feature called nullable reference types. This will help to prevent the potential issues that using null can lead to. Any reference type can be set to null. This simply means that it has not been instantiated. Architecturally, using null can be

ambiguous, and it's often not a good idea. Consider that if we start the process of a set of orders and the method is defined to return a summary of those orders. This returns a reference type, and reference types can be set to null. In this case, if the method returns null, what would that mean? Does it mean that we were unable to process the orders, were there unexpected issues with one or many of the orders, or did the order processing complete but a summary was not necessary for this set of orders and null is used like a representation of a default or empty list? It's really impossible for the consumer of this method to understand what null would mean. There are many other alternative approaches that would have been much better than returning null. Not only could it be ambiguous what null means. If we expect a set of summaries to be returned, if that is null, it could introduce further problems. We'd have to guard against this to check if the result is null before proceeding to look at all these summaries. Hopefully, this convinces you that using nulls is not the best idea. This is where nullable reference types are going to prove extremely handy.

Enable Nullable Reference Types

There is a way to tell the compiler that it should look at reference types as being non nullable. It won't change the code that is generated nor will it prevent you from using null. What this feature will do is help you find the places in the code where a potential null reference exception can occur. As of .NET 6, this is the default behavior, and nullable reference types are enabled by default. For new projects, this makes sense as you'd very much like to avoid nulls and null reference exceptions as much as you can. Each project in a solution can have a configuration that indicates that nullable is enabled or disabled. This is known as configuring the nullable context. This can also be configured using the properties for the project by navigating to Build, General, and then scroll down until you find the section on nullable. Here, you can see the different options for this configuration. We can, of course, enable or disable. But in addition to that, you can configure this to only produce compiler warnings or annotate your code in Visual Studio to indicate that you may have a null reference. This was already set to enable. Does this mean we don't have any potential null reference exceptions? Of course not. If we compile the solution and open the error list and look at the warnings produced by the compiler, notice that it tells us we do have a few potential problems. The nullable setting is per project. And if we were to disable it for each of the projects in the solution, all the warnings would disappear. Let's make sure that it's enabled for all the projects in our solution. Optionally, you can turn on treat warnings as

errors. If we do that, all of a sudden, this would produce compiler errors instead of warnings. This setting is under the errors and warnings in the properties of each project, but can also be configured in the csproj file. The setting is called `TreatWarningsAsErrors`. And when this is enabled, look at what happens if we now compile. These warnings are now errors. If you work in a large application where you transition to using nullable reference types, it may be a little bit overwhelming to treat all of these as errors as you'd have a lot to fix. It can definitely be a good idea to enable this, especially if the code is built continuously in a build pipeline. You'd get immediate feedback about potential issues. We can remove `TreatWarningsAsErrors` for now and go ahead and try to solve these warnings. When we enable nullable reference types, the compiler now assumes that each of the different references that we use in the application will always be set to an instance. If the compiler cannot determine that you've set that to an instance before using it, you will get a warning.

Working with Nullable Reference Types

If we look at the `EventArgs` classes that we introduced earlier in the course, we can see that one of the warnings produced by enabling nullable reference types is that we have not set the property order to an instance. It's telling us that this is a non nullable property, and it cannot determine that this has been set to a value once the class has been instantiated. It doesn't matter if the property is immutable using the `init` keyword. That alone doesn't ensure that this has been set to an instance. One potential way to solve this is to introduce a constructor. When you introduce an empty constructor, the warning moved from the property to the constructor as the compiler now determines that it would be appropriate to instantiate all the non nullable members of the class in the constructor. To do that, you'd have to require the order to be passed as a parameter. There are no longer any warnings in the class. However, we changed how this class is used throughout the application. So if we compile this, we may get a compiler error where this is used because we no longer support an empty constructor. The type is only used in a couple of places. Going around and fixing those errors may not really be an issue. Instead of setting the property when creating the `EventArgs`, we'd expect it to be passed to the constructor. Although `was null` really an issue inside this class, the null reference exception would have occurred where the order property was accessed. While it would definitely be a good idea to always make sure that we set our reference types to an instance, changing the contract of how this class is now constructed may be too big of a change. Let's go to the other `EventArgs` class that has a

similar warning and look at an alternative approach. For the Order property, the compiler hints that we could make this nullable, just like you may have previously done when creating a nullable value type. You can now explicitly indicate that a reference type can be nullable as well. If you indicate that the order property is a nullable order, the warning goes away. We didn't fix any potential null reference exceptions, but we've now told the compiler that anyone that uses this property needs to make sure that it's not null. Let's go to the program and make sure that we have an event handler. Inside this handler, we'll use the event args and access the Order property. Then, we'll get the order number from the instance. Look at that. We have a warning. This is because the compiler knows that order can be null. We'll look at different ways of guarding against null in the next clip. But for now, let's just use the null conditional operator. This helped us prevent a potential null reference exception. We still have a lot of warnings, and I'd recommend that you pause the video and try to fix these on your own. Sometimes, it's okay that a reference is set to null as long as the intention is not ambiguous. In the OrderProcessor, these delegates and event handlers on the processor are optional. And thus, we might just allow these to be null. The compiler will then tell us if we use these without guarding against null. Everywhere in the application where you use a reference type, you may declare that it's nullable. Keep in mind all reference types are now determined to be non null by default. If we make a call to the Process method and explicitly define that we are talking about the overload that accepts a list of orders on the OrderProcessor and pass null, the compiler will tell us that this is not appropriate. The method expects this to be instantiated. While the application may still work, the compiler has no idea if the method handles this internally. But the whole purpose of nullable reference types is that you can indicate that you do not wish to receive a null reference. If you tell the consumers of the method that you want them to pass a reference, that means you can remove unnecessary null checks. Of course, there's no guarantee that you won't get a null anyways. One of the limitations of nullable reference types is that it won't know if each element in an array is null or not. If we pass an array of orders where each position in the array hasn't been instantiated, the warning disappears, but we can see that we'd still have an issue. It's not a silver bullet. But in most cases, it will provide a great amount of help to determine if there is a potential problem. If we change this up a bit and create an array of orders where this array contains one null reference, of course, this provides a warning because we've declared that we have an array of non nullable orders. We may indicate that this is an array of nullable orders. This will then provide a different type of warning. The method signature and what we pass to

the method is now a mismatch. The method expects an IEnumerable of non nullable order. If we go to the method process, we could indicate that the entire IEnumerable may be a null reference. Or we can define that we require an enumerable, but we don't care if the order is an instance or not. Look at what happened further down in this method. We're now getting warnings that this may cause a null reference exception. This was just an example of how flexible you can be when you're enabling the nullable reference types. Let's revert this method back to indicate that we want an instance of an array, as well as each item in that array needs to be instantiated as well. We don't want to have any null references at all. As mentioned earlier, it may be overwhelming if you are converting an old project to use these nullable reference types. We looked at enabling or disabling this feature on the project level. The fact is that you can do this using a compiler directive as well. If you add the compiler directive nullable followed by your setting, for example nullable enable, nullable disable, or any of the other options available, this can be applied anywhere in the code and applies only to the context that you use it in. You could enable the nullable reference types only for a particular if statement a method, a class, or an entire file. This lets you gradually improve the entire project by looking at one portion at a time. This is a good alternative if you are maintaining an older project. We've seen how to enable the nullable reference types, how this changes the way the compiler looks at our types, and how to indicate the intent of what a reference type is. Next, we are going to talk more about guarding against null and try to avoid null reference exceptions.

Guarding against Null

There are multiple ways of guarding against nulls. The nullable reference types provide a way to get a warning when the compiler determines that the instance may be null. This is a good opportunity to guard against a null reference. Throughout this course, we've used the null conditional operator to indicate that we don't want to call a method or access a property if the object is not set to an instance. For example, the null conditional operator is used in the event handler that receives the order. Since the order is marked as a nullable reference type, the compiler will tell us that this is a good idea to check if it's null before accessing its members. In this case, accessing the order number, which is a GUID using the null conditional operator, means that the result will be a nullable GUID. If we introduce a non nullable string and assign this to the order number .ToString, you can see that since the GUID is a nullable GUID, we'll get a compiler warning. By now, you know that you can use the null conditional

operator to access ToString only when that is set to an instance. However, since we explicitly declared that we want a string that is non nullable, this isn't desirable as these will be a mismatch. We really do not want to end up with question marks all over the code base. Instead, let's go to the root of the problem. Let's get rid of the nullable types and indicate that all of this is non nullable. This gets us back to the issue where it tells us that the order may be null. What our intent is with this code is to only run this entire block if the order isn't null. We can wrap this block in an if statement that asks if the order is not null before entering. We've successfully guarded against null, and the compiler will know that the order is set to an instance. Thus, we no longer need the null conditional operator. This makes the code much more understandable, and we found a potential null reference exception, which we've handled accordingly. We performed the null check using pattern matching. We can introduce a property pattern. This will, as you know, first validate that we have an instance, then look at the property. Again, the compiler knows that we've checked if the object is set to an instance. To prove that the compiler is pretty smart, if we use the var pattern, which simply captures this into a local variable, the warning appears again. This is because it does not validate that we have an instance. On the other hand, if we use the type pattern, it does know that we've checked if it's set to an instance. This also works if we invert the if statement and return if the object is not set to an instance. There are other ways to handle potential null references in C# and act accordingly if you determine that an object is null.

Handling Nulls

In addition to the null conditional operator, you also have the help of the null coalescing operator, null coalescing assignment, and the null forgiving operator. In combination with the null conditional operator, we may want to introduce a default value if one couldn't be retrieved. Let's experiment with this. The order that we've defined using a record uses a few default parameters. Setting a reference type to the default value means that this will be null. This produces a warning because we've defined that this is a non nullable property. Change the ShippingProvider to a nullable ShippingProvider. Going back to the event handler that handles when the order is created, we may now want to use the ShippingProvider. This will give you a nullable ShippingProvider. If you explicitly declare that this is a non nullable ShippingProvider, we'll get a warning. As you know by now, this is because the left-hand side of the assignment defines a non nullable reference type, and it's assigned a nullable reference type. In this scenario, we'd like to create an instance or use an already existing instance of the default

ShippingProvider. We can do this by introducing the null coalescing operator. Using the double question marks, we can say that if the left-hand side of this operator is null, we'll use the right-hand side of this operator. Simply put, if the ShippingProvider on the order that has been passed to this event handler is null, we'll create a new instance. Another way to achieve the same thing is to use the null coalescing assignment. On the left-hand side, we use the variable that we want to ensure has a proper instance. Then, the null coalescing assignment is written using double question marks followed by the equal sign. On the right-hand side of this assignment is what will be assigned to the left-hand side if the variable is null. This is just an alternative way of saying that we want to assign the provider to a new instance if the provider is null. Notice that the compiler is smart about this. If we access something from the provider such as the name, it won't give us any warnings because it knows that we've used the null coalescing assignment above. If we comment that part out, the compiler generates a warning. Again, we made sure that we found and worked around a potential null reference exception. The compiler will not always be able to figure out if you've done the appropriate null checks yourself and already have guarded against null. Let's create a class inside the business layer that validates the ShippingProvider before it's allowed to be used for an order. We can introduce a method inside this class that helps with validation based on a set of criteria. Maybe that method already performs a null check. Will the compiler be able to understand this? Let's create a static method that returns a Boolean, and we'll call this ValidateShippingProvider. This accepts a nullable ShippingProvider. Inside this method, we'll use pattern matching to ensure that we have an instance, as well as the freight cost being above 100. If this is the case, we are going to return true. Otherwise, we'll return false. If we use this method inside the event handler to validate the ShippingProvider before accessing its properties, we know that this has performed a null check but the compiler doesn't. I'd argue that this makes sense. Is there a way to indicate to the compiler that we know better and that this has already been taken care of? There's an operator called the null forgiving operator. The null forgiving operator is written using an exclamation mark. This lets you indicate to the compiler that what it warns about potentially being null has already been checked. You guarantee that this is not null. You know what's going to happen if it's null? It'll throw an exception. This does not only make it harder to find the potential null reference exceptions as the warnings now went away. But you've also opened up for the possibility of this actually throwing a null reference exception. Maybe this next example will convince you that this is definitely not a good idea if we explicitly declare that a type is non nullable.

Setting this to null yields a warning. However, we can promise the compiler that null isn't null. And therefore, we'd get rid of the warning. It's just that. It's a way to get rid of the warning. Most of the time, you definitely do not know better than the compiler. In our case, with the method `ValidateShippingProvider`, we definitely did know that this performed a null check. Isn't there a better alternative than using the null forgiving operator? There certainly is. With the help of the attributes for null state static analysis, we can apply attributes to indicate to the compiler what to do in this particular scenario. Of course, the best approach would have been to get rid of the nullable reference types. In a perfect world, we could have introduced an architecture that worked with instances that correspond with a default value instead of relying on null. This way, we can avoid null reference exceptions. We'll take a look at the attributes for null state static analysis next and see how we can change the method `ValidateShippingProvider` to indicate to the compiler that it already performed a null check.

Attributes for Null-state Static Analysis

The compiler tries its best to track the null state during this static analysis. It can't understand everything. So in some cases, it will not automatically understand that we've already guarded against null or, in some cases, that null would be accepted. As we saw when introducing the method `ValidateShippingProvider`, this method does check that the instance of the type that is passed into the method is not null. When this method returns true, we can be certain that this reference is set to an instance. In theory, the best approach in many cases would have been to stick with non nullable reference types. In reality, that's not always going to work or be the best design. This is why there's a set of attributes. In `System.Diagnostics.CodeAnalysis` that will help us to instruct the compiler when it's performing the null state static analysis. We won't be looking at all the attributes. You can always consult the Microsoft docs to see if there's an attribute that can be used in your specific situation or if you simply want to try out all of the different ones available. The attribute that we are going to use is listed in the post condition category of these attributes, and this is known as `NotNullWhen`. We need to bring in the namespace `System.Diagnostics.CodeAnalysis`. We can apply this attribute to a parameter on the method to indicate when the method returns the given Boolean, we promise that the value passed into the method wasn't null. This is a promise that we've performed a null check. The compiler will not know if we have, in fact, checked this or not. Notice what happens if we simply return true and go back to where this method was called. There's no longer a warning that the provider is potentially

null. Let's revert the method body of `ValidateShippingProvider` to ensure that we do, in fact, have a null check. This gave us a way to signal that we've ensured that the nullable type that was passed into the method if we return true, it will definitely be an instance. We only have to do this because the method `ValidateShippingProvider` accepts a nullable type. Another example of a valuable attribute is to indicate that we do, in fact, allow null to be passed where we expect a non nullable type and that we promise that this will be handled accordingly. Let's take a look at what that might look like. We're going to make a change to the order record. The `ShippingProvider` passed to the record is nullable, and that is only because I want this to be an optional parameter. I'd like to indicate that this is a non nullable type. To do that, remove the question mark. This now gives us a warning for the default keyword, and we could get around this by telling the compiler that we forgive this null reference. The problem here is that if someone else passes null for the `ShippingProvider` parameter, they'd get a compiler warning. Let's get rid of the null forgiving operator. Instead, we are going to indicate that we allow this to be null. Then, we can handle this properly in the body of the record. Introduce a public property with the same name used in the primary constructor. Use `get` and `init` to allow this to be set using the object initializer. Finally, we can initialize this with the value passed to the primary constructor. If that is null, we can use the null coalescing operator to create an instance of the default `ShippingProvider` using a target type new expression. We've now explicitly allowed null to be passed as a default value for the `ShippingProvider`. If it's not passed, it will automatically be set to an instance. This is a great way to communicate your intent. And as you see, if you create an instance of an order and set each parameter, it's no longer complaining about the `ShippingProvider` because we've said that we accept this. Enabling nullable reference types makes your life easier. And with the help of the different attributes, you can communicate the intent of the methods that do have to work with potential null references. As a good example of how this is used in .NET, we can replace the if statement we have in the event handler. This verified that we had an instance. And if we remove this, we've got a warning. Let's add a call to a method on the type `ArgumentNullException` called `ThrowIfNull` and pass the order into this method call. Notice how the warning suddenly disappeared. This is because the method uses the attributes for null state static analysis. If this method call completed, it means the object was set to an instance.

Summary

This module discussed the problem of nullability. And if C# was completely redesigned today, I can almost guarantee that they'd make it even harder for us to use nulls. It's just a matter of time before you run into a null reference exception if you haven't already. By default, any reference type in C# can be set to null. With the help of the nullable reference types, we can indicate that all reference types are now assumed to be non null. In .NET 6, this setting was made the default, and all projects now have nullable reference types enabled. You can turn this setting on in the project settings, and it's a per-project configuration. Together with treat warnings as errors, this makes a powerful addition to your toolkit. If you want to use nulls, you have to explicitly define that this is the intent. You can do that just like you've defined nullable value types previously. You use a question mark when specifying the reference type you are using, for example an order that is allowed to be null. This can now be set to null. And everywhere in the application where you access this, the compiler will try its best to determine if it's been set to an instance or not. You can try your best to guard against nulls either by using if statements, pattern matching, the null conditional operator, the null coalescing operator, or the null coalescing assignment. There are plenty of ways to work around nulls. The benefit of the nullable reference types is that it helps you to find where using nulls in your application may cause a problem. It's not a silver bullet, and the compiler will not always be able to figure out if you've performed a null check. You can use the null forgiving operator to indicate that you have already performed a null check elsewhere when the compiler cannot determine that. Be careful with that though. Try to rely on the attributes for null state static analysis if you end up in that situation. Only use the null forgiving operator as your last resort. Finally, we saw how to use the helper method `ThrowIfNull` on the argument, null exception. This is a great guard against unexpected null references. In the next module, we are going to talk about how to slice an array using the range syntax, how to use an index to get the last element in an array, as well as how to introduce an indexer.

Indexers, Ranges, and Indices

What Is an Indexer

Welcome back to the course C# Advanced Language Features. I'm Filip Ekberg. And now, we'll talk about indexers, which let you use a type as an array. Then, we'll be slicing array using ranges. And finally, we'll talk about indices. Let's begin with the indexer. If you've worked with arrays, you've most certainly used an index to access an element in that array. This is known as an indexer. C# allows any object to introduce an indexer. This is useful if you create a type that represents a collection of items where you want to avoid exposing the underlying data structure. Creating an indexer is simple. You use the property syntax or an expression bodied member syntax to do so. Let's say that we have a class that represents a collection of orders. Internally, this class has an array of orders and you don't want to expose that underlying array, but instead let anyone use this class just like if it was an array itself. The end result we'd like to end up with is to be able to use an instance of the OrderList to access an item in the array stored in the class. We'll start by introducing a property. This will return an order. Then, to indicate that this is an indexer, you use the keyword `this`. Next, you specify the type of index that is used to access one of these orders. Since the data structure is an array, you can say that the index passed to this indexer is of type `int`. Then, we can use an expression bodied member to fetch the correct item from the array and return that. This introduced a read-only indexer. It provides consumers of this class a way to use an instance just like a normal array. However, they would be unable to modify the underlying data structure. As an alternative, you can introduce an indexer that accepts another type when accessing the element. We could say that an indexer accepts a GUID to be used to find one of the orders. Then, inside the expression bodied member, we could use LINQ to find the correct item. The problem with this approach is that whoever uses this as an array expects the lookup to be instant. For a small amount of elements, this won't be an issue. But imagine if the underlying data structure is huge and the search is a bit more complex. Then, when you try to access an element using the indexer, the lookup might take some time. That really goes against the purpose of using an indexer. An alternative approach would have been to expose that functionality as a method, which could then clearly communicate that the lookup might take some time. An example usage of the indexer can be found in .NET. If we look at the dictionary, this uses an indexer that takes the generic `TKey` as an indexer and finds the value in the dictionary based on that. This is a great example of how the indexer can be used. If you build complex types that could be accessed in an array-like manner, introducing an indexer is simple. Next, we are going to look at how we can slice arrays by using the range syntax.

Slicing an Array Using the Range Syntax

If we work with an array of orders, how would you write a piece of code that solves the following scenarios? Only select the 10 first orders, skip the first order and get the rest of the array, get all the items from the middle of the array until the end. We could solve all of these scenarios using LINQ. And while the code would have been readable, it introduces a bit of unnecessary complexity. Instead, C# has a way of slicing an array. This is done using what is known as the range syntax. This is used in conjunction with what is called an index. A range is defined by specifying where to begin and where to end. This is separated by two dots. If you specify a range that only uses two dots, that means the entire array. This is the range operator. If you add an index before the double dots, that indicates where to start. If you want to create a range that represents from the start until the second element, you'd write `..2`. Let's say that we'd want to get a slice of an array that contains integers. We'd like to grab the first two elements into a new slice. To do this, we can use the normal indexer on the array. Instead of specifying the index that you want to get, we can supply a range. The range `..` would give you a slice that matches the entire array. The range `..2` would give you a slice that ends with a second element. In other words, it means it slices until the index is less than 2. Slicing the array of numbers and ending with the index 2 would therefore yield a slice that contains 2 elements, 0 and 1. What is specified in the range are indices. This works exactly like when you use an index with an array. You could create an index that refers to the middle of the array by dividing the length with 2. Explicitly specifying 0 as the starting point is unnecessary. You could remove that as the default is that it starts from 0. When the compiler sees the range syntax, it will generate some code for you. This is really just syntactic sugar and you could write all of this using the `System.Range` if you so like to. This shorthand way of defining a range and use it to slice an array is really handy. Each index in the range can be defined with an integer or use `System.Index`. The `System.Index` can be implicitly converted to an integer. What's interesting with the `System.Index` is that it provides a way to determine that the number you specify as an index can be determined to be from the end of the array. This gives you the opportunity to say give me the last five elements in this array. Normally, you'd have to calculate that yourself based on the length of the array, which requires just a little bit more code. There's even an operator for this specific purpose. This is known as the hat operator. If you want to access the last element of the array, you could retrieve that by using the indexer and passing this index. `^1` means the same as `length - 1`, and it will let you access the last element. Since this is an index, it can be used

with a range syntax. You could say that you want to grab all the elements but skip the last one in the slice. You'd use.. followed by ^1. This means grab all the elements but skip the last one. You're not limited to use this to define or to stop slicing. You can also use this to define that you want to start slicing from the end of the array. Let's get the last five elements out of this array. Writing the equivalent using LINQ isn't really complex, but it requires a little bit more code, and I prefer using less characters that are equally as understandable as it makes it easier to maintain the project. To use the range syntax, you have to work with an array, Span of T, or Memory of T. This means if you have a list of T or an IEnumerable, you have to convert that into an array first. Keep in mind that this may cause unnecessary allocations. List of T does support using an index. This makes sense because it can be implicitly converted to an integer. You're definitely not limited to use this with arrays that represents value types. You could use this in the warehouse management system to select a range of orders in a large array. There's another case where you could use the range syntax, that is if you have a string and you want to select a small part of that string. Using my name as an example, I could substring this using the range syntax to get only the first few characters of my name. The range syntax and the indexer provide syntactic sugar to make it easier to access elements and work with slices of data. We've had ways of achieving a similar result in the past, but this approach requires less amount of code to express the same thing. Let's continue and talk about some memory-efficient data structures that you can use, which may greatly improve performance.

Introducing Span<T>

Being able to slice an array into smaller pieces and only work with a portion of the larger array can be beneficial in many cases. In more complex situations, you may work with buffers or, for example, have large bit arrays where you know the fixed positions that you want to slice off. In these situations, if you were to slice that as a normal array, you'd get new arrays and copies of values or references depending on the content of the array. This can lead to an unnecessary number of allocations and a larger memory footprint. There's a bit of help in .NET that allows for a more memory-efficient approach. We can introduce a generic type called Span of T. The definition of a span is that it provides a type and memory-safe representation of a contiguous region of arbitrary memory. The Span of T supports an implicit conversion from Array of T, which means that if we change the variable type to Span of T, this could be used as a span instead. This implicit conversion will give you an instance of a

Span of T. It doesn't require much computational power or additional memory to do this. The implicit conversion is the same as constructing a new Span of T and pass the array to the constructor. This will give you an object that spans the entire array. Internally, the constructor only stores a reference to the array and doesn't create a copy of it. You could easily validate this by modifying an element in this span after you've created a span that represents the array of data. If you do, the elements in the array and the span will be identical. This proves that it, in fact, points to the array and didn't make a copy. Not only can a span be used to reference an array, but it can also be used to work with unmanaged memory. This makes it very useful and a powerful tool. If we use a data structure that implements IEnumerable of T such as List of T, this would have to be converted into an array before it can be used as a span. It's wise to only do this conversion once and then use a local representation of this span throughout the entire method. The span is stored on the stack, and there are some specific situations where the span cannot be used such as in asynchronous methods and fields in your classes. The span comes with built-in support for creating a representation of a slice. This requires minimal processing power and really just lets you point to the different elements in the span. It doesn't have to copy the entire slice of that array. This is extremely important. Not only is it more performant, but it also comes with greater responsibility. If you were to change the value in the slice, that would also change the value in the span that this was sliced off. The beauty of this is that we can request multiple different slices from the same span, and it wouldn't require much in terms of memory or processing power to do so. A good example of where this is truly useful, if you work with a large byte array and you know that certain ranges of elements throughout this array represents important pieces, you can grab those slices without having to copy each portion. What about if we want to pass a span around to have other methods in the application work with the data and leverage the power of that being a span? From what we've discussed so far, this would greatly improve performance throughout the entire application, especially if we can avoid unnecessary allocations. The span is a ref struct, which is stored on the stack, and this alone comes with a few limitations. The compiler will tell you if you use a span or any other ref struct incorrectly. We can use a Span of T as a parameter to a method as long as it's not an asynchronous method. If the method signature defines that you want a Span of T and you pass an array, it would be implicitly converted to a span. Using spans makes it extremely memory-efficient, especially if we need to use slicing throughout the methods. You can use the range

syntax with a span just like we saw before. The only difference is that it's much more efficient because it's simply pointing to two positions in that original array. This will result in a span over the array.

Using Span<T>

Let's say that our warehouse management system has some custom hardware to monitor the business. How the data is being sent into the application or what the payload contains is irrelevant. We can just assume that the structure of the data is simple. And when it arrives to the application, it's a large stream of bytes. We'd like to validate the payload. To do this, we are going to look at the last 128 bytes. If one of these bytes has the value 1, the validation has failed. Of course, a real signature validator would be a little bit more complex, but we are focusing on the span. Let's introduce a payload validator and add a method that accepts a Span of byte. To grab the last 128 bytes, we can do that by using the range syntax. Don't forget the hat. If you do, it will create a slice starting from the 128th byte until the end. That may result in a much larger slice than you like. So far, we've grabbed a slice of the span, and no extra allocations have been necessary. We can introduce a simple check that iterates through this span to validate the data. Again, in reality, this validation would be a little bit more complex. Let's return true at the end of the method. One potential issue that I'd like to highlight is that since we are working with the span, which is just a pointer to an array, does that mean this method can change the original array? Let's go ahead and grab one of the bytes in this span. Let's then change the value and head over to the program where we can create a byte array that will now be passed into this method. If we run this with the debugger attached, we can see that it changed the value in the array when we're inside the Validate method. If we continue the execution, we can see that after we've exited this method, the original array also changed. While this makes sense, this can be problematic. We might not want that anyone tampers with the data when we pass this around. While we've been memory-efficient by avoiding a copy of the array when slicing that, there's still a bit of improvements to be made. We can't prevent anyone from changing the content of the span. We could change it to be a read-only Span of T, and there's an implicit conversion to read-only span as well so you don't have to change anything where this is used. You can pass an array of bytes into this method, which now promises that it won't change anything on the array. It not only promises that it won't change anything, it couldn't if it so wanted to. As a side note, if the read-only span contains reference types and that reference type has properties that are not immutable, the data would be

changeable. A read-only span only makes the operations on the array read-only. It doesn't mean the elements within the array was automatically turned immutable. As mentioned, the span lives on the stack. It cannot be used as a field in your class, and you cannot use it inside an asynchronous method. Most of the time, this is completely fine. In the situations where this causes a problem for you, you can use another class that was built to complement the Span of T known as Memory of T. This isn't going to be as performant as only relying on the span. If you end up in a situation where the span isn't enough and you have to introduce Memory of T, you should really consult the guidelines to make sure that you do not run in to any performance problems. Using span together with the range syntax makes working with different types of arrays much easier no matter what it contains.

Summary

This module introduced you to indexers, ranges, and indices, all of which will make it easier for you to work with collections of data. We saw how to create an indexer. This will let you use an instance of a class just like an array. While most of us do not write custom list-like data structures where a simple method to access elements isn't enough, being able to create an indexer can be truly helpful when you do need it. We saw a real-world implementation from .NET that clearly uses an indexer. We're talking about the dictionary. This introduces a generic indexer that lets you use any type of key to access elements in the dictionary. It's really important to understand that when someone acts as an element using an index, they expect that to be nearly instant. That's why you shouldn't use the indexer like a lookup, which could take a little bit of time. After looking at the indexer, we introduced the new range operator together with an index. The range syntax lets you define a start and end. These two indices are separated with the double dots. If you do not write any index on either side, it will select the entire array. If you use the hat operator, you can define a range that starts from a number of elements off the end. For example, give us the last 128 bytes in this array. Using the range operator or range syntax together with the Span of T provides a great way of working with slices of data. The span provides a type and memory-safe representation of a contiguous region of arbitrary memory. It can be used with arrays, unmanaged memory, and strings. When performing operations on the span to get additional slices, it requires no additional allocations. It's also a very fast operation. You're not limited to use this with buffers or byte arrays that you have read out of a stream. This can be used to look at a string as a chunk of characters. Then, very easily, slice off pieces from this string, and this doesn't

require any additional allocations. Using this in places where your code spends a lot of time working with and storing temporary new strings may be greatly improving performance and the memory footprint. If you want to promise that your method does not change the original array, you can use the `ReadOnlySpan` instead. Keep in mind that you cannot use spans in methods that are marked as being asynchronous, and they cannot be fields in classes. The spans are ref structs that are allocated on the stack. There is a complement to `Span of T` which you can use in an asynchronous method. This is where you'd use `Memory of T`. Although this can have a performance implication, and it is wise to consult the guidelines if you ever run into a situation where the span is not enough. What we've discovered in this module is a powerful way of working with collections of data. In the next and final module, we are going to talk about cleaning up resources and garbage collection.

Garbage Collection

Introducing Garbage Collection

Welcome back to the course `C# Advanced Language Features`. I'm Filip Ekberg. You've reached the final module of this course. You should be extremely proud of yourself for making it through this entire course. We've covered so many important topics that will make you a better developer. In this final module, we'll continue on that path and talk about how to clean up resources and learn a little bit more about garbage collection. `C#` is a managed programming language. This means that you don't necessarily have to worry much about memory management. In some cases though, you can use components that allocate unmanaged memory. It's wise to follow certain patterns and practices to make sure that you do not have unnecessary allocations stuck in memory. The garbage collector will look for objects in memory that you no longer need. This could be an instance that you created in a method. Once that method completes and the instance is no longer relevant, the garbage will be collected. You do not have to manually force the garbage collector to run. You could and this is only appropriate if you create something that manually allocates unmanaged memory. Instead of forcing the garbage collector to run, we should trust that it cleans up resources for us whenever it determines that it's necessary. We can still run into situations where we'd get what looks like a memory leak. The

term memory leak in .NET can be a little bit ambiguous. What I mean is that we have allocated memory that will eventually be collected by the garbage collector but is living in memory longer than it should. A memory leak can also be objects that are unable to be collected by the garbage collector because they are still referenced. I'm talking about this situation where you have an event handler that holds a reference to an object that is no longer relevant, but it cannot be collected. We saw an example of this in the module on events. We need to make sure that the event handlers are always unsubscribed. Otherwise, the garbage collector will think that the object is still relevant, and it won't clean up that resource. When you terminate the application, the memory will be freed as well. So the term memory leak refers to objects that are allocated in memory longer than they should. Eventually, most of these objects will be collected. For the components that use unmanaged memory, which most streams, bitmaps, and graphical components do, you should follow a very basic pattern to ensure that the memory is cleaned up when you are done. In the warehouse management system, I now want to load all the image data available in a certain directory. I'm going to introduce a method in the Windows application that loads all these files from disk. This method will return an IEnumerable of Bitmap. This requires us to install a NuGet package. We can ask Visual Studio to do this for us. Then, let's call this LoadImages. This method will look through a directory that I have added to the project. Each item in this directory will be copied to the output directory. For each file, we are going to read all the bytes. To create a bitmap out of this, we need to pass a MemoryStream. We can construct a MemoryStream that uses the byte array for its data. The bitmap will use unmanaged resources. And if not used correctly, we may end up having a bit of a memory issue. This code does not follow the best practices for how to work with streams of data or bitmaps. We'll improve this soon. But first, we need to understand why this may be a problem. Let's use yield to create an iterator. That way, we do not have to create a temporary list of bitmaps that we later return. This is a cleaner way of producing a stream of data. We can now call this method. Let's do this in the click event handler. Then, let's iterate through the collection of bitmaps and print the size of each bitmap to the debug window. Let's run the application with the debugger attached. Let's make sure that we open up the diagnostic tools in Visual Studio. This will show us the memory usage. Each time the button is pressed, the amount of memory increases. It looks like it never recovers. The application continues to use more and more memory. Eventually, some of this allocated memory may be collected. Depending on the application, your code may use additional memory that you simply cannot afford. If you process a lot of data, doing it like this

may lead to memory being allocated for longer than it should. You may even run out of memory if you're not cleaning up after yourself. You don't know when the garbage collector will start collecting and you shouldn't run it yourself. There is a better way to approach this. So how do we clean up after ourselves in this situation? Each resource that uses unmanaged memory or needs to clean up after itself should implement the disposable pattern. Bitmaps, streams, and other types that want to clean up when they're done implement something known as `IDisposable`. To indicate that we are done with our resource, we can introduce the `using` keyword. This can be used in two different ways, a `using` statement or a `using` declaration. The `using` declaration will ensure that the method `Dispose` is invoked when the method has completed. In our case, we have two resources that both implement `IDisposable`, the `MemoryStream` and the `Bitmap`. The `MemoryStream` can be cleaned up right after we've used this to construct the `Bitmap`. We can assume that the `Bitmap` used the data that it needed to create its own internal structure. If we only dispose of the `MemoryStream` by adding a `using` declaration, it won't have much of an impact. The big problem that we saw with allocations and the memory usage comes from using the `Bitmap`. Since the consumer of this method expects a stream of `Bitmaps`, if we dispose of the `Bitmap` inside this method, anyone that uses this may end up in a little bit of a pickle. They won't be able to use the `Bitmap` because it's been disposed and destroyed. All unmanaged resources will have been cleaned up. Instead, it's very important that whoever consumes this stream of `Bitmaps` will also be in charge of cleaning up after themselves. In the `foreach` loop where we use the `Bitmap`, we can add a `using` statement to define that we've used this `Bitmap`. We can go ahead and dispose of managed and unmanaged resources. With this very small change, let's rerun the application. Now, it doesn't matter how many times you spam the button. The memory usage looks much better. You no longer have allocations in memory that are no longer necessary. This is why it's extremely important to clean up your resources. We'll also look at implementing the `IDisposable` interface later. You can also introduce a `finalizer`. A `finalizer` is called when the garbage collector is ready to collect the instance. Remember, you don't know when this is going to happen. You can use the `finalizer` to make sure that resources are being cleaned up. But in most cases, you want to rely on the disposable pattern instead and introduce the `IDisposable`. This gives the consumer of the class the capability of determining when to clean up what could be expensive resources. It's good to keep in mind that the `finalizer` will only be called when the garbage collector is ready to collect the instance. This means if you have an event handler leak and the instance cannot be collected, you may be left

with allocations that you do not need. The syntax for the finalizer is simple. It looks like a constructor, but it's prefixed with a tilde. You can introduce a finalizer on an object in the application and experiment with this on your own and see when this is executed. Next, we are going to look at how to implement `IDisposable`.

Using `IDisposable`

We saw that because the stream and the bitmap implements `IDisposable`, it can be used together with a using statement or using declaration. This will ensure that the managed and unmanaged resources are cleaned up properly. Doing so will hopefully avoid unnecessary allocation and what we call a memory leak. Implementing the interface `IDisposable` is simple. You can do this on any class, and all that it requires is that you implement a public void method called `Dispose`. This method will be called through the using statement or using declaration after it's done. I'd like to introduce a new class in our business layer of the warehouse management system. Its purpose is to load a batch of orders from a local JSON file, then process each order using the `OrderProcessor`. Let's create the `FileProcessor` class. It will use the `OrderProcessor`, so it's a good idea to pass this to the constructor. This means the `FileProcessor` doesn't have to be in charge of creating that instance. When an order that's loaded from disk has been processed, we'd like to handle the result, sort of like reporting on the progress. To do this, the `OrderProcessor` exposes an event called `OrderCreated`. We can subscribe to this and print a message to the console. To initiate the order processing, we'll do that from a method called `start`. This will read all the text from my file that's included in the project and then deserialize this into a proper array of orders. We can then iterate through all the orders and process them one by one. Would you say that we are done now? You may already have spotted an issue with this class. We attach an event handler to the event exposed by the `OrderProcessor`. But because we never unsubscribe from that, we'll be left with unnecessary allocations that cannot be cleaned up. One way to solve this would have been to subscribe and unsubscribe in the method `start`. However, in some cases, this might not be the only method that's relying on the events. This means you'd have to introduce a lot of duplication. That's not an ideal scenario. Instead, we are going to implement `IDisposable`. This requires the method `Dispose` to be added. In here, we can unsubscribe from all the events and remove any resources that may need to be cleaned up. For example, if you have a stream that's available through the entire class, you could make sure that it's disposed in this method. It's also where you'd clean up

allocated unmanaged memory. This pattern allows you to clean up resources when the instance is no longer useful. Visual Studio will not warn you about missing using statements or using declarations. Implementing IDisposable is not a guard against memory leaks. It's just a pattern to follow to allow you to easily have a way of cleaning up resources. We've now ensured that the Dispose method unsubscribes from all the events that we subscribe to. This class is no longer tied to the OrderProcessor. If we go to the program and create an instance of the FileProcessor in a new method and pass an instance of an OrderProcessor into that, we have to make sure that we also introduce a using statement or a using declaration. You can have multiple using statements or using declarations in the same method. When it comes to the using declaration, the Dispose method on the IDisposable object will execute when the method completes. When you debug this, you will see that the Dispose method executed when the entire method completed. This course does not cover asynchronous or multithreaded programming. That alone is an entire course itself. I do have a course here on Pluralsight covering this subject which I highly recommend. While we are talking about disposable, I do think it's worth mentioning that you can also have an asynchronous disposable. If you implement the interface IAsyncDisposable, the DisposeAsync could of course call the normal Dispose method inside of it and then clean up resources that need to perform their operations asynchronously. To use the AsynchronousDisposable, you append the await keyword in front of the using. What happens if you have multiple using statements for the same variable? Will it execute the Dispose method more than once? Give that a try and see what happens. If your Dispose method shouldn't be called more than once and doing so could introduce problems, you need to keep track if it's already been disposed. We've now seen how to properly dispose of an object by following this very simple pattern, implement IDisposable whenever you need to clean up resources. I cannot stress this enough. Make sure that you always unsubscribe from your events. If the IDisposable doesn't work in your case, you'd have to find something else such as a method that executes when a view disappears. You cannot use the finalizer to rely on your resources being cleaned up. The finalizer will only be called if the garbage collector determines that it can collect the instance. Let's see what happens if we have a finalizer on the FileProcessor and then force the garbage collector to run and simply print something to the console. When the garbage collector takes care of this instance, we should see something printed in our console. If we remove the using statement, it won't reach the finalizer. And by now, you should know why. Add a using, which then cleans up the resources. After this, the finalizer should execute.

This shows why you can't clean up certain resources in the finalizer. It's time to head on to the final piece of this course and talk about boxing and unboxing.

Boxing and Unboxing

When you take a value type and cast that into an object, you've boxed the value. This implicit conversion transfers the value type from the stack onto the heap to store that in an object. This process isn't cheap. It requires additional memory and processing. In high-performance applications, unnecessary boxing can negatively impact the performance. The same goes for unboxing. If boxing means that you implicitly convert a value type to an object, you may have guessed what unboxing means. When you take an integer, decimal, or other value type and pass that into a method that accepts an object, this boxes the value type and converts it into an object stored on the heap. This object will then hold the value. This implicitly converted a value type to the object. To go back to the value type, you have to explicitly declare which value type is stored in the object. If you try to unbox to an incorrect value type, you will get an exception because it cannot cast it into an incorrect type. It doesn't perform any conversions more than wrapping and unwrapping the value type in the object. When you explicitly cast the object back to a value type, the value is extracted from the object and put back onto the stack. This unboxed the value. Do your best to avoid unnecessary boxing and unboxing. Keep this in mind if you work with libraries that accept objects instead of providing a generic alternative.

Summary

In this final module, we talked about garbage collection and patterns that you should follow to reduce unnecessary allocation and what commonly is referred to as a memory leak. If you fail to clean up your resources, you may run into a situation where the application continues to use more and more memory. While the garbage collector may eventually catch up and collect those unused allocations, it may be too late, and the application may have already run out of memory. We saw how this was a problem when we failed to dispose of a bitmap after we were done with it. Another situation is when you forget to unsubscribe to event handlers when your class is done. This will have your instance of that class attached to the event handler, and the garbage collector will not understand that it's no

longer interested in that instance. Therefore, it's crucial to unsubscribe to events when your instance is no longer relevant. There are multiple different ways to solve both of these issues. You can use the dispose pattern and introduce the using statement or using declaration. These will ensure that the Dispose method from the IDisposable is executed when the resource is no longer being used. For the using declaration, the Dispose method is executed when the method completed. You can also use the IDisposable if you need to asynchronously clean up resources. If you are unable to use the IDisposable or that it doesn't fit in your current situation, one example is navigating between views in an application. You can instead clean up your event handlers and potential resources in methods such as disappearing or closing. We also talked about that you shouldn't rely on a finalizer because that will only execute when the garbage collector runs and collects the instance. If your instance happens to be allocated for longer than expected, you may again run into a situation where you have managed or unmanaged resources that are allocated for longer than they have to. We saw an example of the finalizer never being invoked because our class failed to unsubscribe from the event. This meant that the garbage collector thought the instance was still relevant. We knew that it wasn't. Finally, we talked about boxing and unboxing, which is the process of converting a value type between the stack and the heap. If you box a value type, that means it will be wrapped in an object and stored on the heap. To get this back into a value type, you have to explicitly cast it to the correct type. This will unwrap the value from the object and put it back on the stack. Both of these require additional allocations and can lead to performance problems. If you rely on generics whenever you can, you may never have to run into any boxing or unboxing performance issues. However, if you use classes or components that accept an object and don't have a generic alternative, know that every value type passed into the method requires a bit of overhead. This module has given you an insight into what to keep in mind when working with resources that need to be cleaned up. It will help you to avoid memory issues in your applications. And you saw firsthand how this could become a problem.

Final Words and Thank You

A huge congratulations on making it to the end of this course. We've covered so many great and important language features. These advanced language features will make you a better developer. It will help you to understand why you may run into problems, how to properly work with more advanced concepts, and what to keep in mind when building real-world applications. By now, I really hope that

you have a good understanding of how all of this works. And if you liked this course, I would be very happy if you left a rating and checked out some of my other courses. If you've got any questions, I'm here to help. You've just completed C# Advanced Language Features. I'm Filip Ekberg, and thank you for participating in this course.